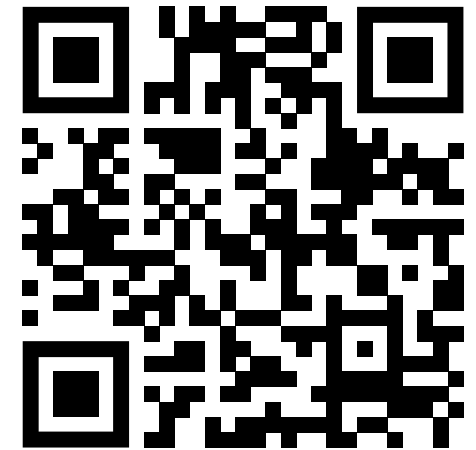


Programmierung in C++

Grundlagen C & C++ (Voraussetzungen)

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>



Grundlagen C & C++

Lernziele

- Am Ende des Kapitels sollen die Grundlagen der Programmiersprachen C und C++ als Voraussetzungen für Programmieren 2 bekannt sein.
 - Die Grundlagen werden in diesem Kapitel kurz erklärt.
 - Das Vorwissen aus Programmieren 1 wird vertieft.
- Die vorausgesetzten Konzepte sollen verstanden und verinnerlicht sein.
 - Das Übungsblatt 0 wird einige der Voraussetzungen wiederholen.
 - Grundlegende Lücken sollen eigenständig von den Studierenden nachgearbeitet werden.

Grundlagen C & C++

Servus Welt!



C

```
#include <stdio.h>

int main()
{
    printf("Servus Welt!\n");
}
```

- Die Programmierkonzepte von C sollten bekannt sein

C++

```
#include <iostream>

int main()
{
    std::cout << "Servus Welt!\n";
}
```

- Die meisten Konzepte von C lassen sich auch in C++ anwenden
- Auch, wenn die C-Syntax meist wiederverwendet werden kann, stellt C++ einige verbesserte Funktionalitäten zur Verfügung

Grundlagen C & C++

Übersicht



- Objekte & Variablen
- Funktionen & Objektübergaben
- Gültigkeit & Lebenszeit
- Programmstruktur
- Zeiger, Arrays, Referenzen
- Laufzeitkonstanten
- Typumwandlung
- Bedingungen & Schleifen
- New & Delete
- Struct, Union
- Enumerationen
- Operatoren



Objekte & Variablen

Lernziele

- **Objekte** wurden im Kontext von Variablendefinitionen verstanden
- Die Objekt**deklaration**, **-initialisierung** und **-zuweisung** können identifiziert und unterschieden werden
- **Moderne Konstrukte** zur Objektdeklaration und -initialisierung können genutzt werden

Objekte & Variablen

Datentypen

■ Deklarationen

führen Entitäten in einem Programm ein und spezifizieren ihren Datentyp

- Ein **Datentyp** definiert die möglichen Werte und Operationen für ein Objekt
- Ein **Objekt** ist ein Speicherbereich, der den Wert eines bestimmten Datentyps enthält
- Der **Wert** ist eine Menge von Bits, die entsprechend dem Datentyp interpretiert werden
- Eine **Variable** ist ein benanntes Objekt



Hier noch keine Objektorientierung!

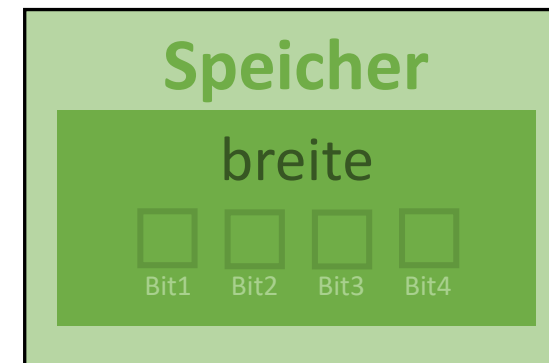


Datentyp: **int**

Variablenname: **breite**

```
int breite;
```

Deklaration der Variable „**breite**“



Objekt / Variable „**breite**“ im Speicher

Objekte & Variablen

Datentypen – Elementar



- **Elementare Datentypen** haben eine feste Größe, die bestimmt, welche Werte gespeichert werden können
 - Der Sprachstandard definiert einen **Mindestwertebereich** der Datentypen

Datentyp	Wertebereich (mindestens)	→ Mindestgröße (Bits)
char	[-127, +128]	8
short	[-32767, +32768]	16
int	[-32767, +32768]	16
long	[-2147483647, +2147483648]	32
long long	[-9223372036854775807, +9223372036854775808]	64

- Die tatsächliche **Bitgröße** elementarer Datentypen ist nicht im Sprachstandard definiert und hängt vom genutzten Compiler ab
 - In der Ära von 16-Bit Betriebssystemen war **int** 16-Bit und **long** 32-Bit groß
 - Daher die Einführung von **long long** für 64-Bit große Integer
 - Auf modernen 64-Bit Betriebssystemen ist **int** 32-Bit groß
 - **long** ist allerdings nur bei GNU-Compilern auf 64-Bit gewachsen, auf dem MSVC-Compiler wurde die Bitgröße von 32-Bit beibehalten

Objekte & Variablen



Datentypen – Elementar – Standard Integer

- Oft werden Integer jedoch in **fester Bitgröße** benötigt, wie bei ...
 - der **Kommunikation** über ein Netzwerk zwischen verschiedenen Rechnern und Betriebssystemen,
 - der **Serialisierung** in Dateien und deren Austausch zwischen verschiedenen Systemen,
 - ...
- Der **<stdint>** (C-Standard-Integers) Header (übernommen aus C) definiert dazu Typ-Aliase für elementare Datentypen fester Bitgröße:

<stdint>	
int8_t, int16_t, int32_t, int64_t	Vorzeichenbehafteter (signed) Integer mit exakt 8, 16, 32 oder 64 Bit
uint8_t, uint16_t, uint32_t, uint64_t	Ganzzahl (unsigned) Integer mit exakt 8, 16, 32 oder 64 Bit

Objekte & Variablen



Datentypen – Elementar – Standard Integer

- Weiter definiert der `<cstdint>` Header Aliase für **Optimierungen** hinsichtlich **Performanz** und **Speichergröße**, die jedoch weniger häufig zum Einsatz kommen:

<cstdint>	
<code>int_fast8_t, int_fast16_t,</code> <code>int_fast32_t, int_fast64_t,</code> <code>uint_fast8_t, uint_fast16_t,</code> <code>uint_fast32_t, uint_fast64_t</code>	Integer mit mindestens 8, 16, 32, 64 Bit und der performantesten binären Darstellung auf dem jeweiligen System
<code>int_least8_t, int_least16_t,</code> <code>int_least32_t, int_least64_t,</code> <code>uint_least8_t, uint_least16_t,</code> <code>uint_least32_t, uint_least64_t</code>	Integer mit mindestens 8, 16, 32, 64 Bit und der kleinstmöglichen binären Darstellung auf dem jeweiligen System

- Diese Aliase erlauben dem Compiler eine performantere binäre Darstellung der Integer zu wählen, die aber trotzdem mindestens die spezifizierte Bitgröße erfüllt

Objekte & Variablen



Datentypen – sizeof & size_t

- **sizeof**(*Datentyp*) gibt die Größe eines Datentyps in Bytes zurück

```
int16_t variableName0 = 350;  
// sizeof(variableName0): 2 (bytes)  
uint_least8_t variableName1 = 100;  
// sizeof(variableName1): 1 (byte)
```

- Der Rückgabewert der sizeof()-Funktion hat den Datentyp „size_t“
- „size_t“ ist ein Integer ohne Vorzeichen, der die Größe des größtmöglichen Objekts im Speicher darstellen kann
 - Ideal für die Indizierung und Längenmessung von Arrays, Zeigern und Containern
 - Größe und Wertebereich sind Compiler-abhängig und entsprechen der Prozessorarchitektur
 - `uint32_t` [0, 4294967265] auf 32-Bit Prozessoren
 - `uint64_t` [0, 18446744073709551615] auf 64-Bit Prozessoren

Objekte & Variablen



Datentypen – ptrdiff_t & Literale

- „ptrdiff_t“ ist ein Integer mit Vorzeichen, der die Differenz zweier Zeiger darstellen kann
 - Ideal wenn Differenz- oder Indexoperationen auch negative Werte liefern können
 - Wie size_t: Abhängig von Compiler und Prozessorarchitektur (int32_t, int64_t)

```
char array[5];  
for (ptrdiff_t i = 4; i >= 0; --i)  
    std::cout << array[i];
```

„i“ wird nach letztem
Durchlauf negativ

- Zur Verbesserung der Lesbarkeit können lange Literale mit einfachen Anführungszeichen (') unterteilt werden

```
double pi = 3.14159'26535'89793'23846'26433'8329'50288;
```

Objekte & Variablen

Initialisierung & Zuweisung



- Die **Initialisierung** ist die Festlegung eines Werts bei der Objekterstellung
 - Zur Initialisierung kann '=' verwendet werden
 - Dabei werden u.U. **implizite Typumwandlungen** durchgeführt
 - **C++11** Alternativ eine **Initialisierungsliste** mit geschweiften Klammern
 - Statt impliziten Typumwandlungen, die zu Informationsverlusten führen können, werden Fehler geworfen

```
double d1 = 2.3;    // d1 ist 2.3
int i1 = 4;         // i1 ist 4
int i2 = 7.8;       // i2 ist 7 (implizite Typumwandlung: double → int)
int i3 { 4 };       // i3 ist 4
int i4 { 7.8 };     // ERROR: double → int Umwandlung
```

- Die **Zuweisung** ist die Änderung eines Werts bei einem bestehenden Objekt

```
int i1 = 4;    // Initialisierung
int i2 { 4 };  // Initialisierung (ab C++11 bevorzugt)
i2 = 5;        // Zuweisung
```

Objekte & Variablen

Initialisierung & Zuweisung – Best Practice



- Variablen sollten immer initialisiert werden
 - Die getrennte Deklaration und spätere Zuweisung schafft Raum für Programmierfehler

```
Texture* tex;

tex = loadFromFile("Wall.png");
if (t == nullptr) return -1;

tex->draw();
```

Deklaration der Variable „tex“ ohne Initialisierung



Variable **tex** kann versehentlich vor der Zuweisung genutzt werden
→ Crash oder schlimmeres

```
Texture* tex { loadFromFile("Wall.png") };
if (t == nullptr) return -1;

tex->draw();
```

Deklaration und Initialisierung der Variable „tex“



Variable **tex** kann erst verwendet werden nachdem sie initialisiert wurde

Objekte & Variablen



auto

- **C++11** Das **auto** Keyword leitet den Datentyp bei der Initialisierung automatisch ab

```
auto b = true;      // b ist ein bool
auto ch = 'x';      // ch ist ein char
auto i = 123;       // i ist ein int
auto d = 1.2;       // d ist ein double
auto b2 {true};     // b2 ist ein bool
```

- **auto**-deklarierte Variablen sind **Typ-sicher!**
 - Der Typ der Variable kann nach der Initialisierung nicht mehr verändert werden
 - Somit ist auto kein dynamischer Datentyp, wie in dynamischen Programmiersprachen (Python, JavaScript, PHP, Perl, Ruby, ...)
- **auto**-deklarierte Variablen müssen bei der Deklaration initialisiert werden
 - Da sonst ihr Datentyp nicht festgestellt werden kann

```
auto i = 123;       // i ist ein int
i = "Hallo";        // ERROR: Typ kann nach Initialisierung nicht verändert werden
auto x;             // ERROR: Muss initialisiert werden
```

Objekte & Variablen



auto – Best Practice

- Durch die automatische Typ-Auflösung können Programmierfehler vermieden werden [Meyers14]
- Ein Vorteil von **auto**-deklarierten Variablen ist, dass diese den Programmierer bei der Deklaration **zwingen** eine Variable zu initialisieren
 - Dadurch werden Programmierfehler vermieden, durch die nicht-initialisierter Speicher verwendet wird (*undefined behavior*)

```
bool isEven;

if ((x % 2) == 0)
    isEven = true;

std::cout << isEven;
```

result wird nicht initialisiert und enthält Speichermüll vom Stack

Ausgabewert ist *nicht-definiert* für den Fall, dass x ungerade ist

```
auto result;

if ((x % 2) == 0)
    result = true;

return result;
```

Durch **auto** entsteht ein Compilerfehler, wenn die Variable nicht initialisiert wurde

[Meyers14] : Scott Meyers, Effective Modern C++, 2014

✖ C3531 'result': a symbol whose type contains 'auto' must have an initializer

Objekte & Variablen



decltype

- **C++11** Die **decltype** (*declaration type*) Funktion gibt den Datentyp eines Objekts zurück
 - Sie wird vom Compiler vor Programmstart ausgeführt
 - Sie kann anstelle der Angabe eines Datentyps verwendet werden

```
int a = 5;  
double b = 5.5;  
  
decltype(a) c = 10;    // c ist vom Typ int (wie a)  
decltype(b) d = 10.5;  // d ist vom Typ double (wie b)
```

- Mit **decltype** kann auch der Datentyp ganzer Ausdrücke ermittelt werden

```
int a = 5;  
double b = 5.5;  
  
decltype(a + b) result; // result ist vom Typ double (da a + b vom Typ double ist)  
result = a + b;
```


Objekte & Variablen



Zusammenfassung

- Eine **Variable** ist ein Speicherbereich (Objekt) mit einem bestimmten Datentyp und enthält einen Wert
 - Wert und benötigte Bitgröße hängen vom Datentyp ab
- Elementare Datentypen haben einen **Mindestwertebereich**
 - Tatsächliche Bitgröße ist Compiler-abhängig, kann aber mit Typen des <stdint> Headers einheitlich festgelegt werden
- Das Setzen eines Werts einer Variablen kann eine **Initialisierung** oder **Zuweisung** darstellen
 - **Initialisierung**: Setzen des Werts bei der Objekterstellung
 - **Zuweisung**: Setzen des Werts eines bereits vorhandenen Objekts
- **C++11** Das Keyword **auto** kann anstelle eines Datentyps verwendet werden und entspricht dem Typ des zugewiesenen Werts
 - Die Initialisierung eines Objekts wird erzwungen, wodurch Programmierfehler vermieden werden können
- **C++11** Mit der **decltype** Compilerfunktion kann der Datentyp einer (anderen) Variable oder eines Ausdrucks angegeben werden

Objekte & Variablen

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Eine Variable / ein Objekt enthält immer einen Wert.
- Der Datentyp definiert die Bitgröße eines Objekts.
- Elementare Datentypen belegen auf unterschiedlichen Systemen immer gleich viel Speicherplatz.
- Eine Variablendeklaration beinhaltet den Datentyp, Name und Wert eines Objekts.
- Die Initialisierung mit einer Initialisierungsliste (geschweifte Klammern) kann nur während der Deklaration einer Variable erfolgen.
- Variablen sollten möglichst immer zum Zeitpunkt der Deklaration initialisiert werden.
- Das **auto**-Keyword ermöglicht die mehrmalige Zuweisung einer Variable mit verschiedenen Datentypen.
- Die **decltype**-Funktion kann den Datentyp eines Ausdrucks bestimmen.

■ Welches sind valide Bitgrößen des Datentyps „**int**“ nach dem Sprachstandard?

- 8-Bit
- 16-Bit
- 32-Bit
- 64-Bit



Funktionen & Objektübergaben

Lernziele

- Ein Überblick über **Funktionen** und ihre Bestandteile wurde geschaffen
 - Funktions**aufrufe** wurden verstanden und können mit kommenden Kapiteln in Zusammenhang gebracht werden
- Ein Verständnis für **Arten der Objektübergabe** (an Funktionen) wurde geschaffen
 - Objektübergaben können zielgerecht anhand von Überlegungen zum **Verwendungszweck** definiert werden

Funktionen & Objektübergaben



Funktionen

- Funktionen spezifizieren die Operationen eines nutzerdefinierten Prozesses
 - Eine Funktion muss **deklariert** sein, um aufgerufen werden zu können – diese Deklaration nennt man auch „Funktions**prototyp**“
 - Die Deklaration besteht aus:
 - **Rückgabety**p der Funktion (*void*, wenn kein Objekt zurückgegeben wird)
 - **Name** der Funktion
 - Anzahl und Typen der **Argumente**, die beim Funktionsaufruf übergeben werden müssen

Deklarationen / Prototypen

```
Element* next_element(); // Funktion ohne Argumente; Gibt einen Pointer auf Element zurück
void exit();             // Funktion ohne Argumente; Gibt nichts zurück
double square(double);   // Nimmt ein Double als Argument; Gibt ein Double zurück
```

Definition

```
double square(double d) { return d * d; }
```

Aufruf

```
double s2 = square(2); // Ruft square() mit dem Argument double{2} auf
double s3 = square("three"); // FEHLER: Argument vom Typ "const char*" ist mit Parameter vom Typ "double" inkompatibel
```

Funktionen & Objektübergaben



Objektübergaben

- Als **Objektübergabe** wird die Zuweisung des Werts eines Objekts auf ein anderes Objekt bezeichnet
- Ein Objekt kann grundsätzlich auf zwei verschiedene **Arten** übergeben werden
 - **Call-by-value**: Der Wert des Objekts (= value) wird direkt übergeben und dabei kopiert
 - **Call-by-reference**: Der Wert des Objekts wird indirekt über eine Referenz übergeben

In C++ wird die Art der Objektübergabe immer explizit definiert und ist von keinen Compilerregeln abhängig!

Vergleich Java, Python, ...: „Call by value where value could be a reference“ ☹️

Funktionen & Objektübergaben



Begriffsunterscheidung

- Objektübergaben erfolgen bei allen Zuweisungen von Werten zwischen Variablen
 - Parameterobjekte einer Funktion werden wie Variablen initialisiert
 - Variablen werden auch „by-value“ oder „by-reference“ initialisiert
- Diese Vorlesung betrachtet Funktionsaufrufe und Variableninitialisierungen gleichbedeutend hinsichtlich der Objektübergabe
 - Die Namen „Call-by-value“ und „Call-by-reference“ implizieren historisch-bedingt nur einen Funktionsaufruf (*function call*)

```
void function(int x2) { /*Anweisungen*/ }

int main()
{
    int x1 = 5;
    function(x1); // Übergabe Funktionsaufruf
}
```

Übergabe x1 → x2

=
=

Gleicher Maschinencode
(ohne Funktionsrumpf)

```
int main()
{
    int x1 = 5;
    int x2 = x1; // Übergabe Variable
    ... // Anweisungen
} Übergabe x1 → x2
```

Funktionen & Objektübergaben



Call-by-value

- Wird ein Objekt ohne Zeiger und ohne Referenz übergeben, so wird das Objekt *by-value* übertragen
 - Es wird ein **neues Objekt** erstellt
 - Der Wert des einen Objekts wird in das andere **kopiert**

```
void function(int x2)
{
    x2 += 2;
    std::cout << "x ist: " << x2;
}
```

x ist: 7

x2 ist ein neues Objekt vom Typ int

Neues Objekt wird geändert

```
int main()
{
    int x1 = 5;
    function(x1);
    std::cout << "x ist: " << x1;
}
```

x ist: 5

x1 bleibt nach Funktionsaufruf gleich

Funktionen & Objektübergaben



Call-by-reference

- Wird ein Objekt als Zeiger oder als Referenz übergeben, so wird das Objekt *by-reference* übertragen
 - Es wird kein neues Objekt erstellt
 - Das übergebene Objekt wird **referenziert**

```
void function(int& x2)
{
    x2 += 2;
    std::cout << "x ist: " << x2;
}
```

x ist: 7

x2 ist eine Referenz auf ein Objekt vom Typ int

Referenziertes Objekt wird geändert

```
int main()
{
    int x1 = 5;
    function(x1);
    std::cout << "x ist: " << x1;
}
```

x ist: 7

x1 wurde durch Funktionsaufruf geändert

Äquivalent mit Zeiger:

```
void function(int* x2)
{
    *x2 += 2;
    std::cout << "x ist: " << *x2;
}
```

x ist: 7

```
int main()
{
    int x1 = 5;
    function(&x1);
    std::cout << "x ist: " << x1;
}
```

x ist: 7

Funktionen & Objektübergaben



Value vs. Reference

- Häufig stellt sich bei der Ausformulierung von Funktionen die Frage, ob ein Objekt als **Wert** (Call-by-value) oder **Referenz** (Call-by-reference) übergeben werden soll
- Wichtige Überlegungen bei der Objektübergabe sind:
 - Wird ein Objekt **gemeinsam** mit der Funktion **genutzt** (shared)?
 - Wenn ein Objekt gemeinsam genutzt wird: Wird es durch die Funktion **verändert**?
 - Wie **teuer** ist der **Kopiervorgang** des Objekts?

Funktionen & Objektübergaben



Value vs. Reference – Veränderliche Objekte

- Wird ein Objekt gemeinsam mit der aufgerufenen Funktion genutzt und von dieser **verändert**, wird eindeutig eine Referenzübergabe benötigt

Beispiel: Inkrementierungsfunktion

```
void increment(int& x2)
{
    x2 = x2 + 1;
}

int main()
{
    int x1 = 5;
    increment(x1);
    std::cout << "x ist: " << x1;
}
```

x ist: 6

Objekt wird durch
Inkrementierungsfunktion
genutzt und verändert
→ Call-by-reference

Funktionen & Objektübergaben



Value vs. Reference – Nicht-veränderliche Objekte

- Wird ein Objekt nicht gemeinsam mit der aufgerufenen Funktion genutzt, d.h. von dieser **nicht verändert**, kann das Objekt kopiert werden

Beispiel: Ausgabefunktion

```
void printInt(int x2)
{
    std::cout << "x ist: " << x2;
}

int main()
{
    int x1 = 5;
    printInt(x1);
}
```

x ist: 5

Objekt wird durch
Ausgabefunktion genutzt
aber **nicht** verändert
→ Call-by-value

Funktionen & Objektübergaben



Value vs. Reference – Kopierkosten

- Bei allen Objekten, die als Kopie (Call-by-value) übergeben werden können, stellt sich die Frage: **Wie teuer ist der Kopiervorgang des Objekts?**
 - „teuer“ in diesem Kontext: Zeitintensiv
- Als Daumenregel gilt hierbei:

Objekte, die eine Größe von **zwei bis drei Zeigern** haben, sind billiger zu kopieren als zu referenzieren

[Stroustrup22]

- Das schließt alle elementaren Datentypen (int, long long, float, double, Zeiger, ...) ein – diese sollten nie aus Performanzgründen als Referenz übergeben werden!

[Stroustrup22] : Bjarne Stroustrup, A Tour of C++, 2022

Funktionen & Objektübergaben



Value vs. Reference – Kopierkosten

Exkurs: Warum keine Referenzen bei kleinen Objekten?

- Referenzen werden vom Compiler i.d.R. mit Zeigern realisiert
- Zeiger führen einen „Umweg“ über den Speicher ein

→ Variablen werden nicht mehr direkt auf den Stack gepusht, sondern müssen erst aus anderen Speicheradressen geladen werden (Ausblick: Speicherverwaltung)

C++

```
void printInt(int i)
{
    std::cout << i;
}
```

```
void printIntRef(int& i)
{
    std::cout << i;
}
```

ASM

```
main:
    push [i] ; direkte Kopie auf Stack
    call printInt
printInt:
    pop eax ; direkte Kopie aus Stack
    call cout, eax
```

(stark vereinfacht)

```
main:
    lea ebx, [i] ; Push der Speicheradresse (Zeiger)
    push ebx
    call printIntRef
printIntRef:
    pop ebx
    mov eax, [ebx] ; Laden aus Speicher (Zeiger)
    call cout, eax
```

(stark vereinfacht)

Zusätzliche
Operationen
und
Speicherzugriff

Funktionen & Objektübergaben



Value vs. Reference – Konstante Referenzen

- Objekte, die kopiert werden können, allerdings **teuer zu kopieren** sind, sollten als **konstante Referenzen** übergeben werden

Beispiel: Ausgabefunktion eines großen Objekts

```
void printString(const std::string& x2)
{
    std::cout << "x ist: " << x2;
}

int main()
{
    std::string x1 = "Dies ist ein langer Text ...";
    printString(x1);
}
```

x ist: Dies ist ein langer Text ...

Objekt wird durch
Ausgabefunktion **nicht** verändert,
ist aber teuer zu kopieren
→ Call-by-reference to const

Wichtig: **const**!
Sonst ändert sich die
Interpretation des Parameters
(Objekt wird veränderlich)

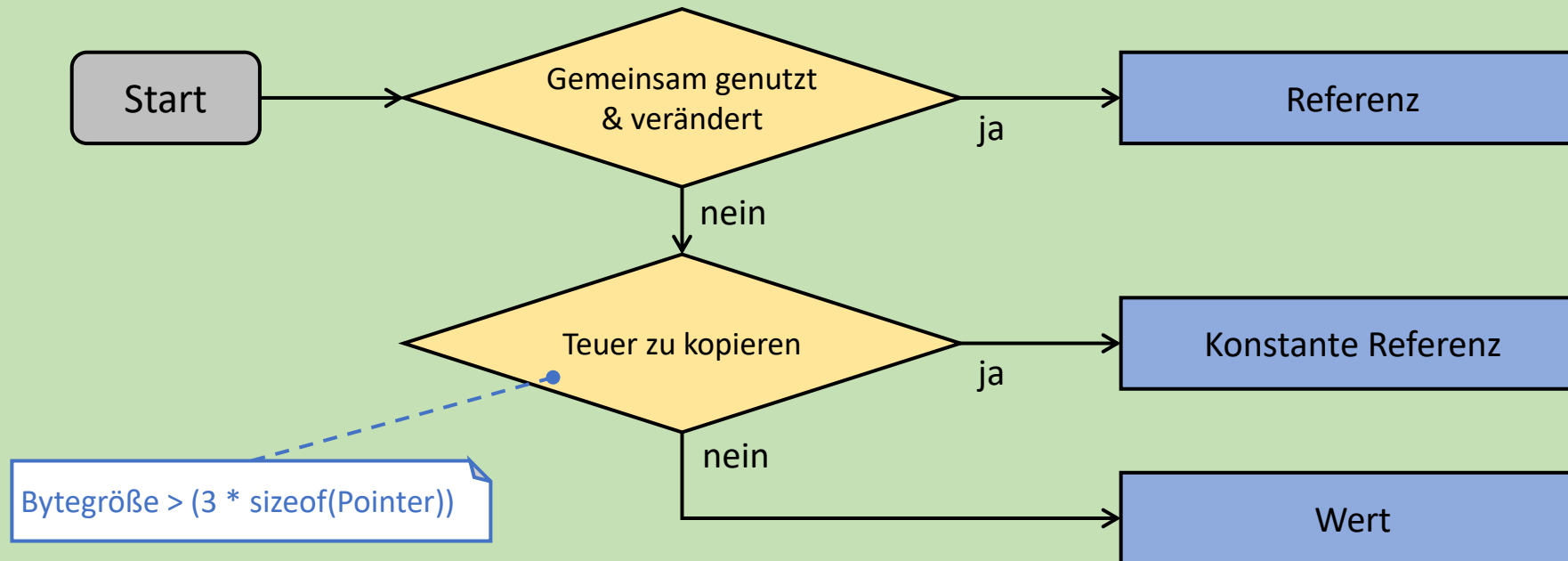
Hinweis: Ein String enthält ein **char**-Array, das u.U. sehr
groß und damit teuer zu kopieren ist!

Funktionen & Objektübergaben



Zusammenfassung

- Funktionen führen Anweisungen aus; können dazu Objekte entgegennehmen und zurückgeben
- Objekte können direkt als **Wert** (Call-by-value) oder indirekt als **Referenz** (Call-by-reference) übergeben werden
 - Die Wahl der Übergabeart hängt von Kriterien ab:



Funktionen & Objektübergaben

PIN: 469452

<https://poll.hs-kempten.de/poll/>



Verständnisfragen

■ WAHR oder FALSCH?

- Ein Funktionsprototyp ist die Deklaration einer Funktion mit ihrem Namen, Argumenten und Rückgabewert.
- Bei „Call-by-value“ kann der Originalwert einer Variable im Funktionsaufruf verändert werden.
- Mit „Call-by-reference“ kann Speicher gespart werden, wenn große Objekte übergeben werden sollen.
- „Call-by-reference“ ist nur mit Referenzvariablen (mit **&**-Symbol deklariert) möglich.
- Die Art der Objektübergabe ist abhängig vom Datentyp einer Variable (ohne Zusätze).
- Bei Objektübergaben an Funktionen sollten grundsätzlich Referenzen verwendet werden, da diese schneller sind.

■ Welchen Übergabetyp hat das Funktionsargument „x“?

```
void increment(int* x);
```

- Zeigerobjekt: by-reference
- Zeigerobjekt: by-value
- Integerobjekt: by-reference
- Integerobjekt: by-value



Gültigkeit & Lebenszeit

Lernziele

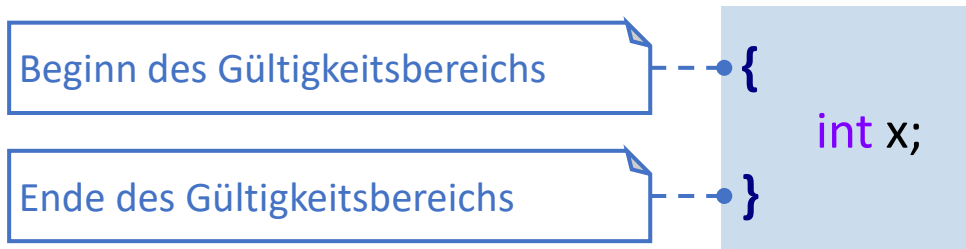
- Die **Geltungsbereiche** von Variablen wurden verstanden
- Die **Lebenszeit** einer Variable kann anhand ihrer Deklaration bestimmt werden
 - Dazu können verschiedene Arten von **Speicherdauern** der Variablen unterschieden werden
 - Eigene Variablen können hinsichtlich ihrer Lebenszeit korrekt definiert werden

Gültigkeit & Lebenszeit

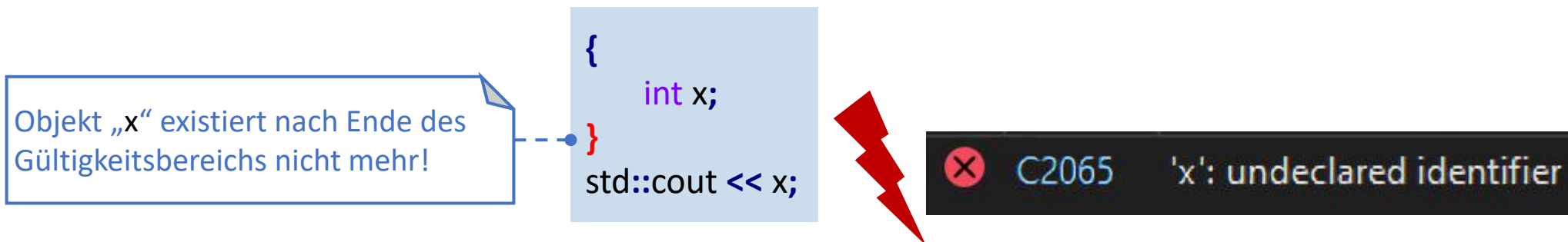


Gültigkeitsbereiche / Scopes

- Ein Gültigkeitsbereich (= **Scope**) wird mit geschweiften Klammern definiert
 - Ein Gültigkeitsbereich kann auch ohne vorhergehende Anweisung oder Deklaration (z.B. Funktionskopf, IF-Anweisung, ...) aufgespannt werden



- Ein Objekt / eine Variable ist nur in ihrem umschließenden Scope gültig



Gültigkeit & Lebenszeit



Gültigkeitsbereiche / Scopes

- Werden Gültigkeitsbereiche unter einer bestimmten Anweisung oder Deklaration aufgespannt, definieren sie den Inhalt der Anweisung oder Deklaration
 - Beispiel: Inhalt einer Funktion (= Funktionsblock)

```
int main()  
{  
    int x;  
}
```

Objekt „x“ gehört zur
Funktion „int main()“

- Beispiel: Inhalt des TRUE-Falls einer IF-Anweisung

```
if (rand() % 2 == 0)  
{  
    std::cout << "Gewonnen!";  
}
```

Ausgabe gehört zur IF-Anweisung
und wird ausgeführt, falls die
Bedingung „true“ ergibt

Gültigkeit & Lebenszeit



Lebenszeit

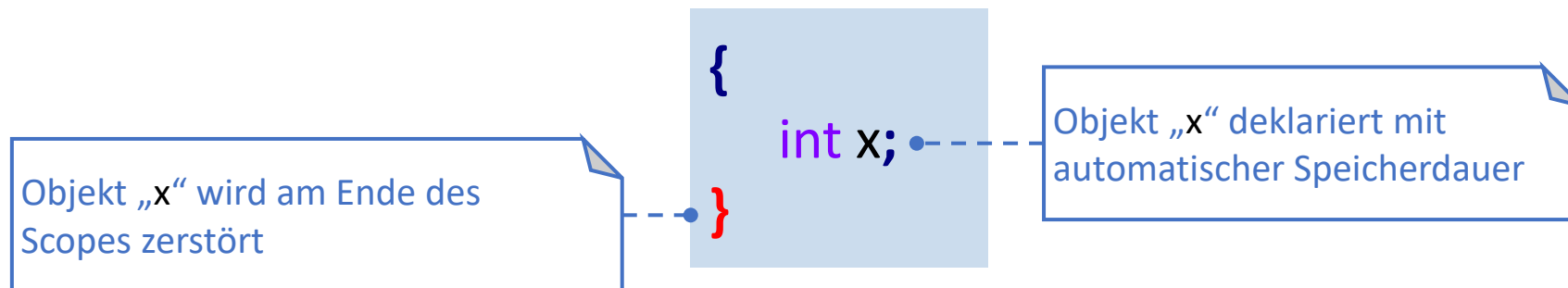
- Unter „**Lebenszeit**“ eines Objekts wird betrachtet, wie lange ein Objekt nach seiner Erzeugung existiert
- Entscheidend für die Lebenszeit ist der **Speicherbereich**, in dem ein Objekt beheimatet ist
- Unterschieden wird zwischen fünf Arten der **Speicherdauer** (*storage durations*)
 - **automatisch**: Das Objekt lebt so lange, wie es in seinem umschließenden Scope gültig ist
 - **dynamisch / manuell**: Das Objekt lebt so lange, bis es manuell durch den Programmierer gelöscht wird
 - **statisch**: Das Objekt lebt so lange, bis das gesamte Programm beendet wird
 - *temporär*: Das Objekt lebt so lange, bis es nicht mehr von Ausdrücken verwendet wird (führt hier zu weit)
 - **C++11 thread**: Das Objekt lebt so lange, bis der eigene Thread terminiert wird (führt hier zu weit)

Gültigkeit & Lebenszeit



Lebenszeit – automatisch

- Objekte mit **automatischer** Speicherdauer leben so lange, wie sie in ihrem umschließenden Scope gültig sind
 - Standardverhalten für Funktionsvariablen
 - Das **Ende eines Scopes** kennzeichnet die Zerstörung des Objekts
 - Viele moderne Sprachkonstrukte machen sich diese Speicherdauer zu Nutze (Ausblick: RAII)



Gültigkeit & Lebenszeit

new & delete



- Mit dem **new**-Operator kann ein Objekt dynamisch erzeugt werden
 - Das Objekt wird zur Laufzeit – also nach Start des Programms – im Speicher erstellt
 - Das erstellte Objekt wird über einen Zeiger verwaltet
- Mit dem **delete**-Operator werden Objekte gelöscht, die mit dem **new**-Operator erzeugt wurden
 - Damit wird das Objekt zerstört und der Speicher, den das Objekt belegt, wieder freigegeben

```
// Erzeuge ein Integer-Objekt und  
// initialisiere es mit dem Wert 3:  
int* intPtr = new int(3);  
  
std::cout << intPtr << ": " << *intPtr;  
  
// Zerstöre das Integer-Objekt  
delete intPtr;
```

Ausgabe:

00000289975B0260: 3

Ausblick: Speicherverwaltung

Gültigkeit & Lebenszeit



Lebenszeit – dynamisch / manuell

- Objekte mit **dynamischer** Speicherdauer leben so lange, bis sie manuell durch den Programmierer zerstört werden
 - Das Ende eines Scopes hat keine Auswirkung auf die Lebensdauer des Objekts
 - Der Programmierer trägt die volle Verantwortung über die Lebenszeit des Objekts

Das Zeiger-Objekt „p1“ wird am Ende des Scopes zerstört, **nicht** aber das darin befindliche dynamische Integer-Objekt!

```
int* p2;  
{  
    int* p1 = new int(3);  
    p2 = p1;  
}  
delete p2;
```

Ein anonymes Integer-Objekt wird **dynamisch** erstellt und über den Zeiger „p1“ verwiesen

Das dynamische Integer-Objekt wird **manuell** gelöscht

Gültigkeit & Lebenszeit



Lebenszeit – statisch

- Objekte mit **statischer** Speicherdauer leben so lange, bis das gesamte Programm beendet wird
 - Objekte haben eine statische Speicherdauer, wenn sie außerhalb jeglicher Scopes von Funktionen, Klassen oder anderen Strukturen definiert werden
 - Sie werden beim Programmstart (außerhalb von Funktionsaufrufen) erzeugt

Objekt „x“ deklariert als globale Variable mit statischer Speicherdauer

Das Ende des Scopes hat keine Auswirkung auf die Lebenszeit von „x“

```
int x = 5;
```

main.cpp

```
int main()  
{
```

```
    std::cout << "x ist: " << x;
```

```
}
```

```
x ist: 5
```



Die Initialisierungsreihenfolge von Objekten mit statischer Speicherdauer ist nicht definiert!



Gültigkeit & Lebenszeit

Zusammenfassung



- **Gültigkeitsbereiche** (*scopes*) werden über geschweifte Klammern aufgespannt und definieren einen Start- und Endpunkt, in dem Objekte / Variablen gültig sind
 - Sie können alleinstehen oder den Inhalt einer bestimmten Anweisung / Deklaration definieren
- Die Lebenszeit von Objekten ist abhängig von ihrer **Speicherdauer** (*storage duration*)
 - Variablen, die in einem Gültigkeitsbereich / Scope definiert sind, werden an der schließenden Klammer zerstört (**automatische Speicherdauer**)
 - Mit den **new/delete**-Operatoren kann der Programmierer manuell steuern, wann ein Objekt erzeugt oder zerstört werden soll (**dynamische Speicherdauer**)
 - Variablen, die außerhalb von Gültigkeitsbereichen definiert sind, leben bis zum Ende des gesamten Programms (**statische Speicherdauer**)

Gültigkeit & Lebenszeit

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Ein Gültigkeitsbereich / Scope benötigt zur Definition immer eine vorhergehende Anweisung.
- Objekte mit dynamischer Lebenszeit werden zerstört, wenn ihr umschließender Gültigkeitsbereich endet.
- Objekte haben eine statische Lebenszeit, wenn sie außerhalb jeglicher Gültigkeitsbereiche definiert sind.
- Eine Variable, die im TRUE-Block einer IF-Anweisung definiert ist, kann auch außerhalb des Blocks verwendet werden.
- Ein Objekt, das mit dem **new**-Operator erstellt wurde, hat den gleichen Gültigkeitsbereich wie lokale Variablen.

main.cpp

```
int main()
{
    int* p = new int(5);
}
```

■ Welche Speicherdauer haben die Objekte im Beispiel links?

- int(5): dynamisch
- int(5): automatisch
- p: statisch
- p: automatisch
- p: dynamisch



Programmstruktur

Lernziele

- Die Verarbeitung von Quelldateien durch den **Kompilier-** und **Linker-Prozess** wurde verstanden
 - Eigene Objekte können unter Rücksichtnahme des Linkings korrekt **deklariert** werden
 - Linker-Fehler können nachvollzogen und vermieden werden

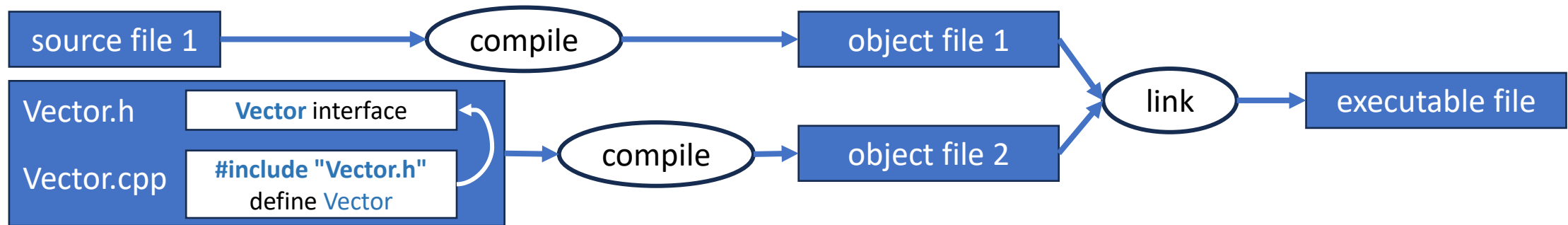
Programmstruktur

Build-Prozess



■ C++ ist eine **Compiler-Sprache**

- Die Quelldateien (.cpp, .h, ...) werden von einem Compiler verarbeitet
- Jede .cpp-Quelldatei wird zu einer Objektdatei (.obj) kompiliert, die die Objekte der .cpp-Datei enthält (Typen, Variablen, Maschinencode, Metainformationen, ...)
- Die Objektdateien werden vom Linker zu einem ausführbaren Programm vereint

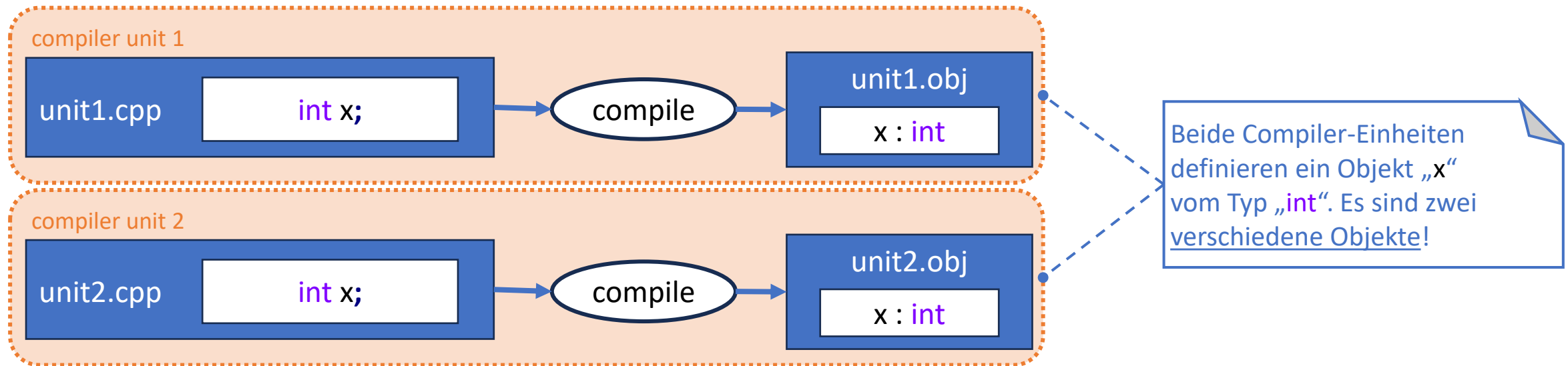


Programmstruktur

Compiler-Einheiten



- Bei den einzelnen .cpp-Dateien und den resultierenden Objektdaten, auf denen der Compiler operiert, spricht man von „**Compiler-Einheiten**“ (**compiler units**)
 - Auch „**Übersetzungseinheiten**“ (**translation units**) genannt
- Compiler-Einheiten kennen sich während des Kompilervorgangs untereinander nicht
 - Definitionen einer Compiler-Einheit sind nicht in anderen sichtbar (keine Transitivität)

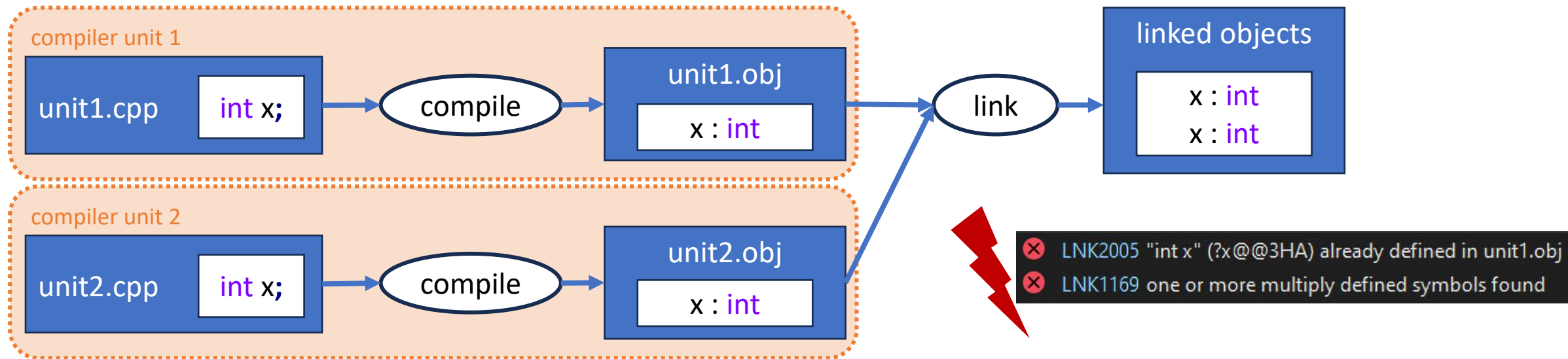


Programmstruktur

Linking



- Nachdem der Kompiliervorgang jeder Compiler-Einheit abgeschlossen ist, verknüpft der **Linker** alle Definitionen der Compiler-Einheiten miteinander
 - Definitionen aller Compiler-Einheiten sind in einem großen Pool sichtbar (transitiv)
 - Definitionen werden anhand ihres Namens identifiziert
 - Definitionen müssen eindeutig benannt sein (**One-Definition-Rule**)
 - Kommt ein Definitionsname mehrfach vor, wird ein **Linkerfehler** ausgelöst

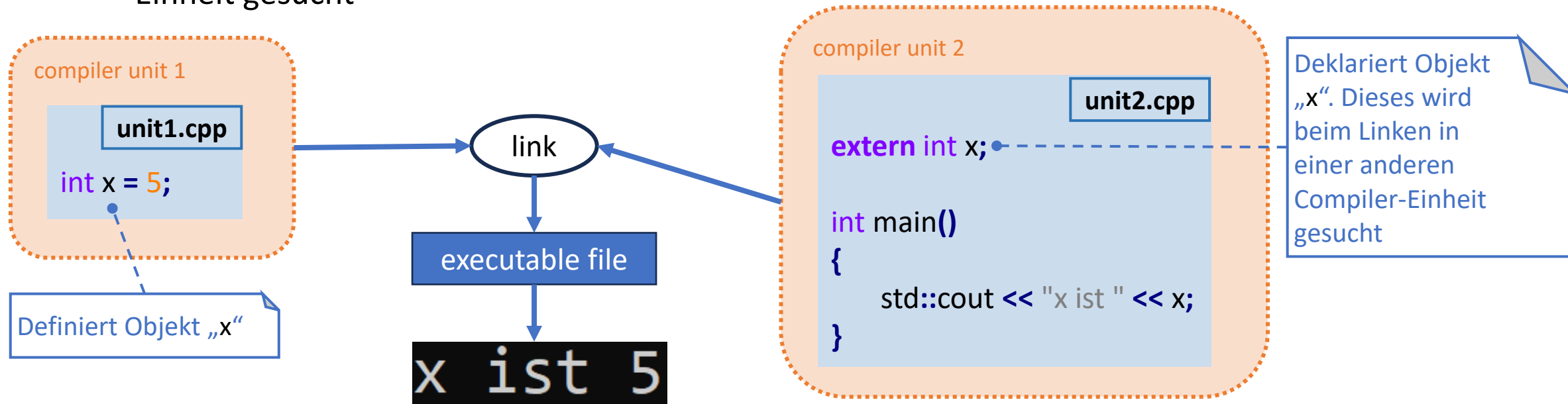


Programmstruktur



Linking – Extern (extern)

- Mit dem **extern** Keyword kann ein Objekt einer Compiler-Einheit von einer anderen verwendet werden (*external linkage*)
 - Ein vorhandenes Objekt wird deklariert
 - Das Objekt wird nicht (mehrfach) definiert und auch nicht dem Linker übermittelt
 - Stattdessen wird beim Linken eine Definition gleichen Namens aus einer anderen Compiler-Einheit gesucht

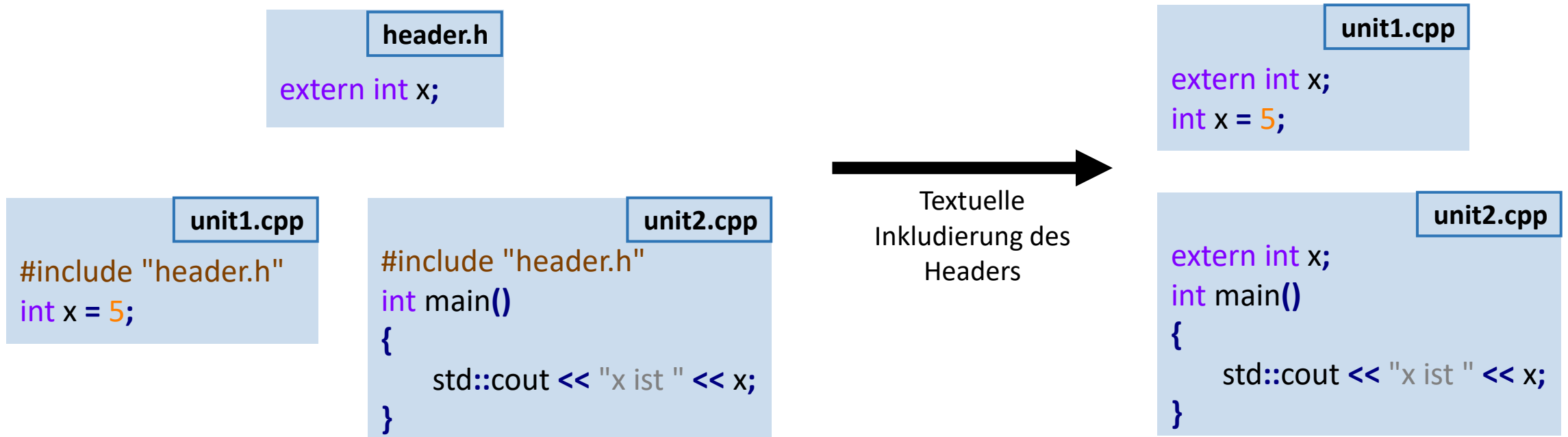


Programmstruktur

Header-Dateien



- **Header-Dateien** dienen der einheitlichen Deklaration von Objekten, die von mehreren Compiler-Einheiten verwendet werden
 - Sie werden rein textuell in die inkludierende Datei (.cpp) eingefügt
 - Jede Deklaration muss von mindestens einer Compiler-Einheit definiert werden, damit sie beim Linken vorliegt und verwendet werden kann

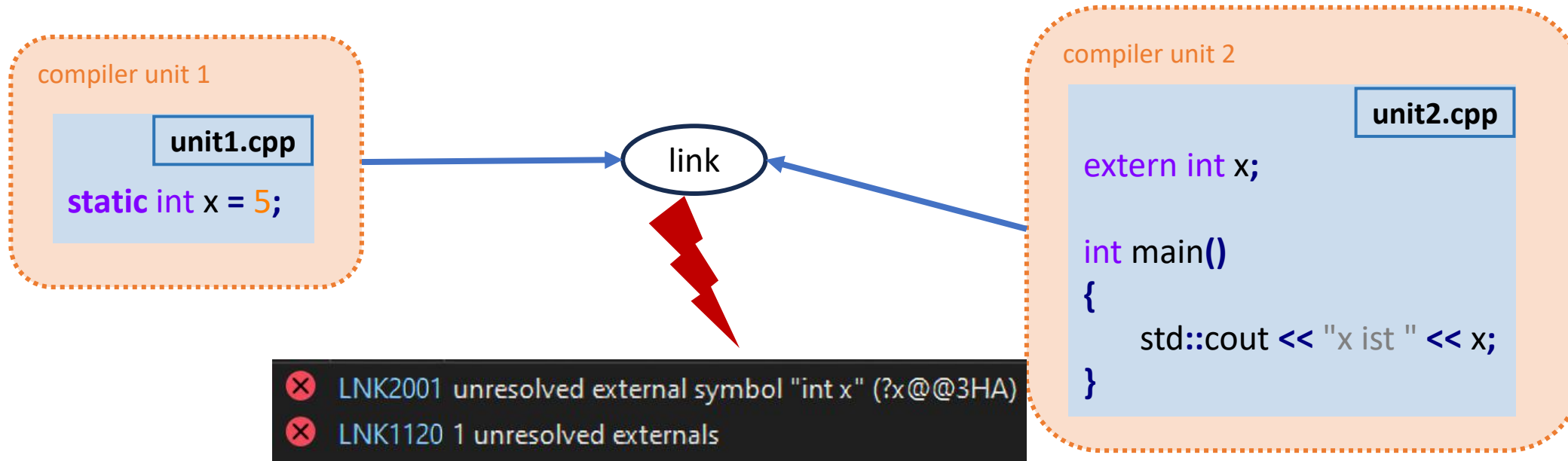


Programmstruktur



Linking – Intern (static)

- Mit dem **static** Keyword wird verhindert, dass ein Objekt einer Compiler-Einheit von einer anderen verwendet werden kann (*internal linkage*)
 - Das Objekt wird vom Linker nicht berücksichtigt
 - Es ist für andere Compiler-Einheiten nicht sichtbar und kann damit auch nicht mehrfach vorkommen



Programmstruktur

Linking – Funktionen



- Im Gegensatz zu Variablen werden **Funktionen** grundsätzlich extern gelinkt
 - Die Angabe des **extern** Keywords ist nicht nötig
 - Funktionen können jedoch auch mit dem Keyword **static** intern gelinkt werden

header.h

```
void normalFunction();  
void staticFunction();
```

unit1.cpp

```
#include "header.h"  
  
void normalFunction()  
{  
    std::cout << "normal";  
}  
  
static void staticFunction()  
{  
    std::cout << "static";  
}
```

unit2.cpp

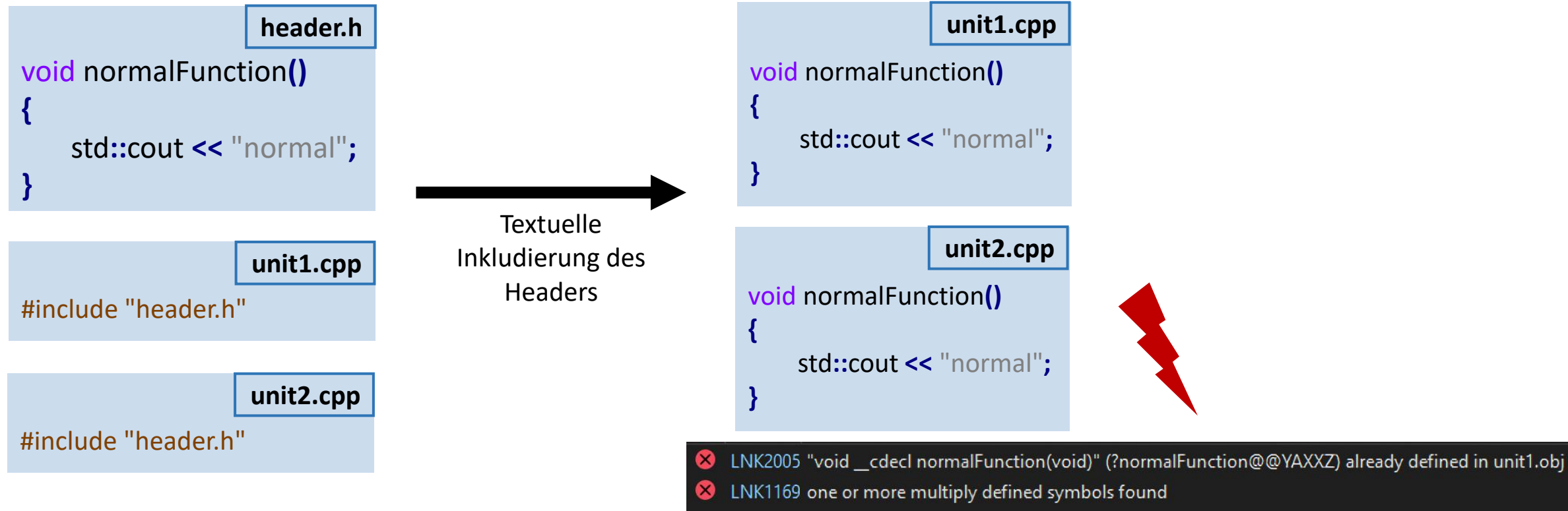
```
#include "header.h"  
  
int main()  
{  
    normalFunction(); // OK  
  
    staticFunction(); // ERROR LNK2019: unresolved  
                      external symbol "void __cdecl  
                      staticFunction(void)"  
                      (?staticFunction@@YAXXZ)  
                      referenced in function main  
}
```

Programmstruktur



Linking – Inline

- Werden Objekte in einer Header-Datei definiert, kommt es zu einer Verletzung der ODR (One-Definition-Rule), sofern mehr als eine Compiler-Einheit den Header inkludiert

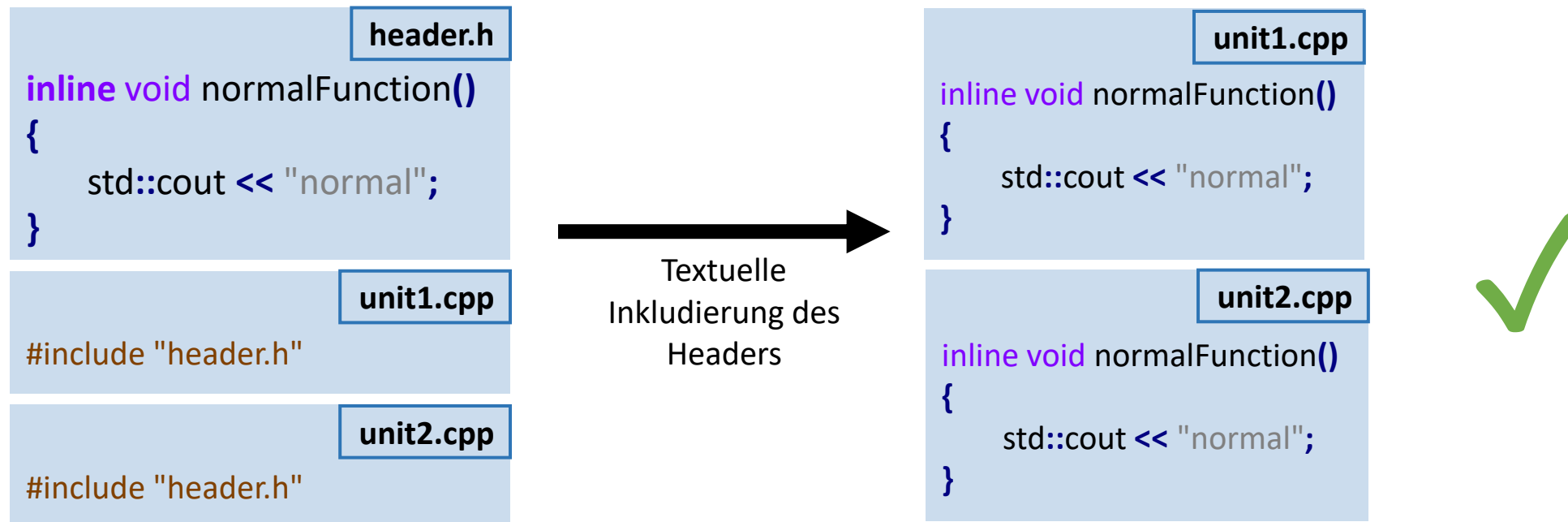


Programmstruktur

Linking – Inline



- Das **inline** Keyword erlaubt die Mehrfachdefinition eines Objekts
 - Das Objekt darf durch mehrere Compiler-Einheiten definiert werden
 - Jede Definition des Objekts muss identisch sein!
 - Z.B. gleicher Funktionsblock bei Funktionsdefinitionen



Vor dem C++98-Standard bedeutete das Keyword **inline**, dass ein Funktionsblock im Maschinencode direkt kompiliert werden soll, statt einen Funktionsaufruf dafür zu bauen. Dies ist nicht zu verwechseln mit der aktuellen Bedeutung des Keywords für den Linker!

Programmstruktur

Linking – Standardverhalten



- Der Sprachstandard definiert für die Art des Linkings ein Standardverhalten, abhängig vom deklarierten Objekt
 - Meist reicht dieses aus, um Probleme beim Linken von Compiler-Einheiten zu vermeiden
 - Objektdefinitionen werden grundsätzlich an den Linker übermittelt, außer sie werden **static** deklariert

	Linkage (ohne Angabe der Keywords extern, inline oder static)
Globale Variable	intern
Funktion	extern
Klasse, Struct	extern & inline
Member-Funktionen	extern & inline
Enumeration	extern & inline
Templates	extern & inline

Programmstruktur

Zusammenfassung



- Quelldateien werden vom **Compiler** zu Objektdaten kompiliert
- Objektdaten werden vom **Linker** zu einem ausführbaren Programm vereint
- Definitionen einer Objektdaten (Variablen, Funktionen, ...) werden vom Linker entsprechend ihrer **Linkage** verknüpft
 - **Intern:** Objekt nur in erzeugender Compiler-Einheit sichtbar, wird nicht zum Linken übermittelt
 - **Extern:** Objekt in allen Compiler-Einheiten sichtbar, darf nur einmal definiert werden (ODR)
 - **Extern & Inline:** Objekt in allen Compiler-Einheiten sichtbar, darf mehrfach definiert werden so lange alle Definitionen gleich sind
- Header-Dateien dienen primär der einheitlichen Deklaration von Objekten

Programmstruktur

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ Welche Linkage hat diese Deklaration?

```
static int square(int x)
```

- Intern
- Extern
- Extern & Inline
- Nichts davon

■ WAHR oder FALSCH?

- Header-Dateien werden vom Compiler direkt zu einem ausführbaren Programm kompiliert.
- Mit **#include** werden andere Compiler-Einheiten in die aktuelle Compiler-Einheit eingebunden.
- Die Objektdaten einer Compiler-Einheit enthält Variablen und Funktionen, die von der Compiler-Einheit definiert wurden.
- Eine Compiler-Einheit kann während des Kompiliervorgangs auf eine andere Compiler-Einheit zugreifen.
- In der Linker-Phase sind alle exportierten Objekte der Compiler-Einheiten untereinander sichtbar.
- Mit dem **inline** Keyword wird in modernem C++ definiert, dass der Maschinencode einer Funktion direkt eingebunden werden soll, ohne dafür einen Funktionsaufruf zu erstellen.
- Eine **extern** deklarierte Variable hat keinen eigenen Speicherplatz und muss in einer anderen Datei definiert sein.



Zeiger, Arrays, Referenzen

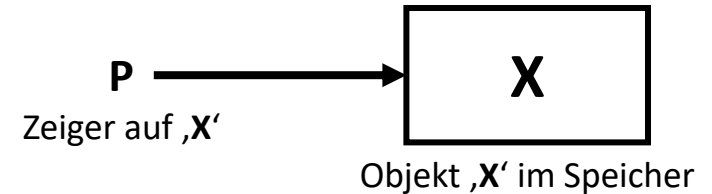
Lernziele

- Die Funktionsweise von **Zeigern**, **Arrays** und **Referenzen** wurde detailliert verstanden
- Die **Arithmetik** von Zeigern wurde im Zusammenhang mit Arrays verstanden
- Zeiger und Referenzen können anwendungsorientiert eingesetzt werden
 - Dazu können die Merkmale und Eigenschaften von Zeigern und Referenzen unterschieden werden

Zeiger, Arrays, Referenzen

Zeiger

- Ein **Zeiger** (Pointer) ist die **Adresse** eines Objekts im Speicher
 - Die Adresse eines Objekts kann durch *Referenzierung* mit dem **&**-Operator erhalten werden
 - Das adressierte Objekt kann durch *Dereferenzierung* mit dem *****-Operator erhalten werden
 - Die Adresse kann numerisch / arithmetisch behandelt werden
 - Kann in- und dekrementiert werden
 - Kann berechnet werden
 - Größe entspricht der CPU-Architektur
 - 32-Bit Integer auf x86,
 - 64-Bit Integer auf AMD64,
 - ...
 - Ein Zeiger kann *nicht-zugewiesen* sein, was mit dem Adresswert **0** definiert wird = **Nullpointer**



```
int* p = 0; // Nicht-zugewiesener Zeiger
```

```
int x = 5;
```

```
p = &x; // Adressierung der Variable 'x'
```

```
std::cout << *p << " an Adresse " << p;
```

```
5 an Adresse 000000782FEFF804
```

```
*p = 10; // Dereferenzierte Zuweisung
```

```
std::cout << "*p: " << *p << ", x: " << x;
```

```
*p: 10, x: 10
```

Zeiger, Arrays, Referenzen

Arrays



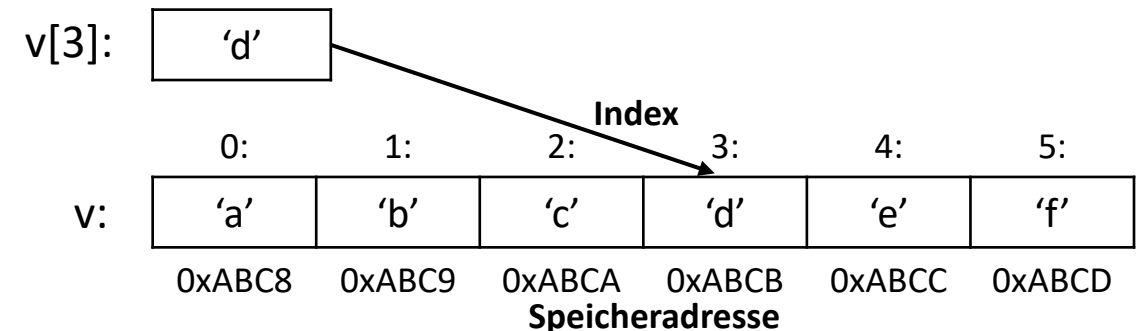
- Ein **Array** ist eine Sequenz von Elementen desselben Datentyps
 - Ein Array wird mit eckigen Klammern `[]` deklariert
 - Die Größe des Arrays muss bei der Deklaration mit angegeben werden
 - Die Elemente des Arrays liegen hintereinanderliegend (*sequenziell*) im Speicher
 - Einzelne Elemente werden mit einem Index über den `[]`-Operator adressiert

```
// Array mit 6 Elementen namens 'v' vom Typ 'char'  
char v[6]; // Annahme: Alphabetisch gefüllt
```

```
// Ausgegeben wird das 4. Element von 'v'  
std::cout << "v[3] ist: " << v[3];
```

```
v[3] ist: d
```

zum Beispiel:





Zeiger, Arrays, Referenzen

Arrays – new[] & delete[]

- Ist die Größe eines Arrays erst zur Laufzeit bekannt, so muss es **dynamisch erzeugt** werden
- Mit dem **new[]**-Operator (eckige Klammern) können Arrays dynamisch erzeugt werden; dabei wird – wie beim **new**-Operator ...
 - ... das Objekt zur Laufzeit – also nach Start des Programms – im Speicher erstellt,
 - ... das erstellte Objekt über einen **Zeiger auf das erste Element des Arrays** verwaltet
- Objekte, die mit dem **new[]**-Operator erzeugt wurden, müssen mit dem **delete[]**-Operator (eckige Klammern) gelöscht werden

```
// Erstelle ein dynamisches Array für drei Integer-Werte  
int* dynamicArray = new int[3];
```

```
for (int i = 0; i < 3; ++i)  
    dynamicArray[i] = i;  
for (int i = 0; i < 3; ++i)  
    std::cout << dynamicArray[i] << ", ";
```

```
// Gib den allokierten Speicher frei  
delete[] dynamicArray;
```

0, 1, 2,

Ausblick: Speicherverwaltung

Zeiger, Arrays, Referenzen

Zeiger – Arithmetik



- Zeiger eignen sich besonders gut, um mit Arrays und verketteten Datenstrukturen zu rechnen
 - Ein Zeiger auf ein Element eines bestimmten Datentyps kann jedes Element des Typs aus einem Array adressieren
 - Elemente des Arrays können durch einfache Additionen und Subtraktionen des Zeigerwerts durchlaufen werden
- Dies wird als **Zeigerarithmetik** bzw. **Pointerarithmetik** bezeichnet

```
// Array mit 6 Elementen namens 'v' vom Typ 'char'  
char v[6]; // Annahme: Alphabetisch gefüllt
```

```
char* p = &v[0]; // Adressierung erstes Element  
std::cout << "p ist: " << *p;
```

p ist: a

```
p++; // Inkrement (p = p + 1)
```

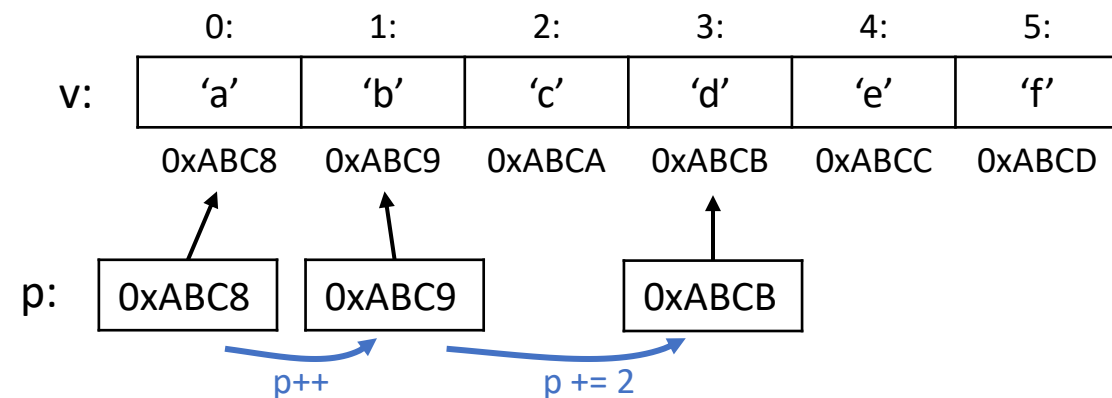
```
std::cout << "p ist: " << *p;
```

p ist: b

```
p += 2; // Addition (p = p + 2)
```

```
std::cout << "p ist: " << *p;
```

p ist: d



Zeiger, Arrays, Referenzen



Zeiger – Arithmetik

- Bei der Addition und Subtraktion mit Zeigern erhöht bzw. verringert sich der Adresswert um die Größe des Datentyps

→ Nicht direkt um den Summanden bzw. Subtrahenden!

```
int* p = 0;  
std::cout << (p + 1);
```

Ausgabe: 00000004

```
long long* p = 0;  
std::cout << (p + 1);
```

Ausgabe: 00000008

```
std::string* p = 0;  
std::cout << (p + 1);
```

Ausgabe: 00000028

Hexadezimal: 28
≙ Dezimal: 40
≙ Größe std::string

Überall $(p + 1)$ → führt aber zu verschiedenen Ergebnissen!

- Bei der Addition und Subtraktion mit Zeigern gilt also:

$(p + X)$



$(p + X * \text{sizeof}(*p))$

„Zeiger plus numerischer Wert X “

„Zeiger plus numerischer Wert X mal Datentypgröße“



Zeiger, Arrays, Referenzen

Zeiger – Arithmetik

- Die Variable eines Arrays ist ein Zeiger auf das erste Element im Array

```
char v[6]; // gleich wie 'p'  
char* p = v; // gleich wie 'v'
```

p wird zu `&v[0]` aufgelöst

- Zeiger und der `[]`-Operator können austauschbar verwendet werden, um Elemente eines Arrays zu adressieren

`v[5]`

„Element an Index 5“



`(p + 5)`

„Zeigerwert plus 5“

p sei `&v[0]`

Fun fact: Da die Addition kommutativ ist, ist auch der `[]`-Operator kommutativ

`(5 + p)`

„5 plus Zeigerwert“



`5[v]`

„5 an Index Element“

Funktioniert! Kurios!

Bitte nicht verwenden!

Zeiger, Arrays, Referenzen



Zeiger – void-Pointer

- Zeiger können auch Speicher adressieren, ohne den konkreten Datentyp zu kennen, der dahinter liegt
 - Diese Zeiger referenzieren den Datentyp **void** und werden als **void-Pointer** bezeichnet

```
void* p;
```

- void-Pointer zeigen auf **jeden möglichen** Datentyp
 - Sie zeigen nicht auf Nichts!
 - Sie können auf jeden Zeiger eines anderen Typs zugewiesen werden
 - Sie führen eine erste Art der **dynamischen Typbehandlung** ein

```
char c = 'a';  
int i = 5;  
float f = 3.14f;
```

```
void* p = 0; // 'p' ist nicht-zugewiesen
```

```
p = &c; // OK: 'p' zeigt auf einen 'char'  
p = &i; // OK: 'p' zeigt auf einen 'int'  
p = &f; // OK: 'p' zeigt auf einen 'float'
```



Zeiger, Arrays, Referenzen

Zeiger – void-Pointer

- Mit void-Pointern kann keine Arithmetik betrieben werden
 - Da die Größe des dahinterliegenden Typs unbekannt ist

```
char v[6];  
void* p = v;  
char v3 = *(p + 3); // ERROR: 'void*': unbekannte Größe
```

- Um mit void-Pointern arbeiten zu können, müssen diese in einen konkreten Datentyp zurückgewandelt werden
 - Genannt „*Interpretation* des Zeigers / Speichers“

```
char v[6]; // Annahme: Alphabetisch gefüllt  
void* p = v;  
char v3 = *((char*)p + 3); // OK: Rückwandlung von 'void*' in 'char*'  
std::cout << "v3: " << v3;
```

v3: d

Zeiger, Arrays, Referenzen



Zeiger – NULL vs. nullptr

- Vor dem **C++11** Standard wurden Zeiger als *nicht-zugewiesen* gekennzeichnet, indem sie auf das **Integer**-Literal **0** gesetzt wurden
 - Häufig gibt es hierzu eine Präprozessor-Definition **NULL**, die zumindest die Lesbarkeit von Nullpointern verbessert

```
#define NULL 0
```

- Die Problematik dabei ist, dass Typauflösungen undurchsichtig werden, die einen **Pointer**-Typen **oder** einen **Integralen**-Typen annehmen
- Der **C++11** Standard definiert daher ein neues Literal „**nullptr**“, das immer einen Pointer-Typen statt einen Integer-Typen darstellt
 - **nullptr** wird bei Verwendung implizit in jeden Pointer-Typen umgewandelt ...
 - nicht aber in integrale Typen!
 - **nullptr** ist ein Literal – wie es auch die Zahl **0** ist

Zeiger, Arrays, Referenzen



Zeiger – NULL vs. nullptr

- Beispiel: Eine Funktion ist *überladen* für einen integralen Parameter und für einen Pointer-Parameter

```
void f(int x) { std::cout << "int,"; }  
void f(void* p) { std::cout << "pointer,"; }  
  
int main()  
{  
    f(0);  
    f(NULL);  
    f(nullptr);  
}
```

Wie lautet die Ausgabe?

Zeiger, Arrays, Referenzen



Zeiger – NULL vs. nullptr

- Beispiel: Eine Funktion ist *überladen* für einen integralen Parameter und für einen Pointer-Parameter

```
void f(int x) { std::cout << "int,"; }  
void f(void* p) { std::cout << "pointer,"; }  
  
int main()  
{  
    f(0);  
    f(NULL);  
    f(nullptr);  
}
```

Integraler Wert: 0

Pointer Wert: Null

Ausgabe:

```
int,int,pointer,
```

→ **nullptr** sollte immer gegenüber **NULL** und **0** bevorzugt werden

Zeiger, Arrays, Referenzen

Referenzen



- Eine **Referenz** ist ein Alternativname (*alias*) für ein Objekt

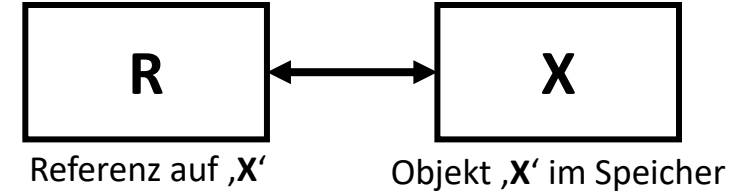
- Referenzen können anstelle einer Variable selbst verwendet werden
- Eine Referenz verhält sich wie ein Pointer, der bei Nutzung automatisch dereferenziert wird

```
int& r = x;  
rx = 42;
```



```
int* px = &x;  
*px = 42;
```

- Referenzen müssen immer initialisiert sein
- Referenzen können nach ihrer Initialisierung nicht mehr verändert werden
- Auf Referenzen ist keine Arithmetik möglich
- Referenzen verweisen immer auf einen konkreten Typen (keine „void-Referenz“)



```
int x = 5;  
int& r = x; // Referenz der Variable 'x'  
std::cout << "x: " << x << ", r: " << r;  
x: 5, r: 5
```

```
r = 10; // Referenzierte Zuweisung  
std::cout << "x: " << x << ", r: " << r << std::endl;  
x: 10, r: 10
```

Zeiger, Arrays, Referenzen

Beispiel



```
int v[] = {0,1,2,3,4,5,6,7,8,9};
```

```
// Kopiere die Elemente von v und gib sie aus
```

```
for (auto x : v) // Für jedes x in v
```

```
    std::cout << x << ',';
```

```
std::cout << '\n';
```

Erstellt eine Kopie für jedes Element.

=> Änderungen auf x wirken sich nicht auf das Original-Array aus.

```
// Erhöhe die Elemente von v um 1
```

```
for (auto& x : v) // Referenz auf die Elemente, um sie ändern zu können
```

```
    ++x;
```

Erstellt Referenz je Element auf das entsprechende Element.

=> Änderungen auf x wirken sich auch auf das Original-Array aus.

```
// Gib die Elemente von v aus
```

```
for (uint8_t idx = 0; idx!=10; ++idx) // Länge des Arrays muss bekannt sein
```

```
    std::cout << v[idx] << ','; // Zugriff über den Index
```

Ausgabe: `0,1,2,3,4,5,6,7,8,9,
1,2,3,4,5,6,7,8,9,10,`

Zeiger, Arrays, Referenzen



Zeiger vs. Referenzen

- Zeiger und Referenzen erlauben den Verweis auf Objekte (Call-by-reference); sie sind in ihrem Verhalten ähnlich, jedoch nicht gleich

	Zeiger	Referenz
Syntax	komplex (&, *)	einfach
Kann direkt anstelle des Objekts verwendet werden	✗	✓
Kann leer sein (null)	✓	✗
Kann nach Initialisierung verändert werden	✓	✗
Kann berechnet werden (Arithmetik)	✓	✗
Kann undefinierte Typen referenzieren (void)	✓	✗

Zeiger, Arrays, Referenzen

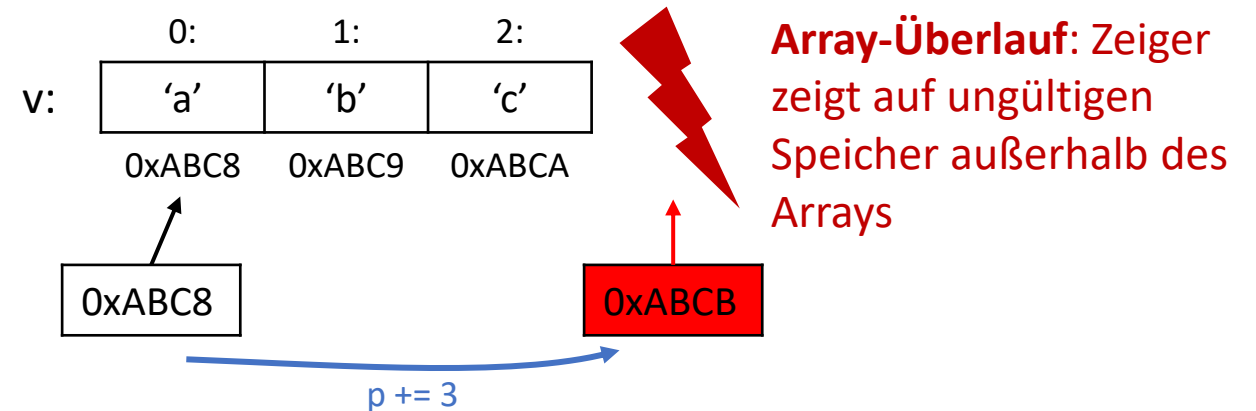


Zeiger vs. Referenzen

- Die vermeintlichen Vorteile von Zeigern schaffen Raum für große **Sicherheitsrisiken** durch **Programmierfehler**, z.B.:
 - Zugriff auf Nullpointer
 - Array-Überläufe = Zeiger auf Speicher außerhalb des Arraybereichs
 - Zeiger auf ungültigen Speicher (Ausblick: Prozessspeicher)
- All diese Fehler führen zu Programmabstürzen oder schlimmerem (Ausblick: Prozessspeicher)

```
char v[3]; // Annahme: Alphabetisch gefüllt
char* p = &v[0]; // Adressierung erstes Element

p += 3; // Addition auf Index 3 = Element #4
*p = 'z'; // Zuweisung des 4. Elements im Array,...
          // das es nicht gibt!
```



Referenzen sollten gegenüber Zeigern (wenn möglich) bevorzugt werden

Zeiger, Arrays, Referenzen

Zusammenfassung



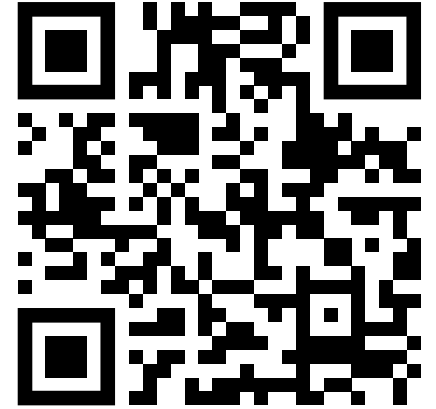
- Ein **Zeiger** ist die *Adresse* eines Objekts im Speicher
 - Wird durch *Referenzierung* / *Adressierung* eines Objekts erhalten und durch *Dereferenzierung* zurückgewandelt
 - Ist ein numerischer Adresswert, der arithmetisch berechnet werden kann
 - Kann *nicht-zugewiesen* sein, was durch den Wert **0** dargestellt wird („Nullpointer“)
 - Kann auf einen beliebigen Datentyp zeigen (void-Pointer)
- Ein **Array** ist eine *Sequenz* von Elementen desselben Datentyps
 - Einzelne Elemente können gut mit Zeigern adressiert und verwaltet werden
- Eine **Referenz** ist ein *Alternativname* für eine Variable
 - Wird durch direkte Zuweisung einmalig gesetzt und kann nicht mehr verändert werden
 - Verweist immer auf ein Objekt mit einem konkreten Datentyp
- Referenzen sind weniger fehleranfällig und sollten gegenüber Zeigern bevorzugt werden

Zeiger, Arrays, Referenzen

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Ein Zeiger ist immer eine numerische Variable.
- Referenzen können dereferenziert werden.
- Das „Dereferenzieren“ eines Zeigers greift auf das Objekt zu, das der Zeiger adressiert.
- Elemente eines Arrays sind zufällig im Speicher verteilt.
- Die Variable eines Arrays speichert die Adresse des ersten Elements im Array.
- Arrays, die mit dem new[]-Operator erzeugt wurden, werden mit dem delete-Operator gelöscht.

```
int v[] = {1, 10, 100};  
int& r = v[1];  
r = r + 1;  
std::cout << r;
```

■ Wie lautet die Ausgabe des Codes links?

- | | |
|------|-------|
| ■ 0 | ■ 11 |
| ■ 1 | ■ 100 |
| ■ 10 | ■ 101 |



Laufzeitkonstanten

Lernziele

- Der **Nutzen** der Deklaration von Laufzeitkonstanten wurde verstanden
- Laufzeitkonstanten können in eigenen Programmen effizient genutzt werden
 - Dazu präsentiert das Kapitel verschiedene Anwendungsbeispiele

Laufzeitkonstanten

Konstanten



- Der Kennzeichner **const** definiert, dass ein Objekt nach seiner Zuweisung zur Laufzeit nicht verändert werden kann (= Konstante)
- Die Definition von Konstanten schützt vor **ungewollten Änderungen** und **Zuweisungen** von Objekten

```
int i1 = 1;  
int i2 = 2;  
if (i1 = i2) // "funktioniert", keine Warnung  
{  
    std::cout << "gleich!";  
}
```

= statt ==

gleich!

```
const int i1 = 1;  
const int i2 = 2;  
if (i1 = i2) // Compilerfehler  
{  
    std::cout << "gleich!";  
}
```

- Auch kann der Compiler – bei der Erstellung von Maschinencode – Konstanten oft optimieren und Berechnungsprozesse beschleunigen

Laufzeitkonstanten



Deklaration

- Für **einfache Variablen** kann der Kennzeichner **const** sowohl vor als auch hinter dem Datentyp angegeben werden

```
const int x = 42; // 'x' ist eine Integer-Konstante
int const y = 42; // Gleich wie oben;
                // 'const' kann vor und hinter dem Datentyp angegeben werden

x += 5; // ERROR: Konstante kann nicht verändert werden
y += 5; // ERROR: Konstante kann nicht verändert werden
```

Laufzeitkonstanten

Deklaration



- Für **Pointer** ändert sich die Interpretation je nach Platzierung des **const** Kennzeichners stark!
 - Wird **const** vor dem Pointersymbol ('*') angegeben, so zeigt der Pointer auf eine Konstante
 - Wird **const** nach dem Pointersymbol ('*') angegeben, so ist der Pointer selbst eine Konstante

```
int x = 5;  
int y = 42;
```

```
const int* p1 = &x; // 'p1' ist ein Pointer auf eine Integer-Konstante  
int const* p2 = &x; // wie p1
```

```
*p1 += 5; // ERROR: Integer-Konstante kann nicht verändert werden  
p1 = &y; // OK: Pointer auf Integer-Konstante kann verändert werden
```

Veränderlicher Pointer auf Konstante

```
int x = 5;  
int y = 42;
```

```
int* const p3 = &x; // 'p3' ist ein konstanter Pointer auf einen  
// veränderlichen Integer
```

```
*p3 += 5; // OK: Integer kann verändert werden  
p3 = &y; // ERROR: Pointer auf Integer-Konstante kann nicht verändert werden
```

Konstanter Pointer auf veränderliches Objekt

- Auch die Kombination ist möglich:

```
const int* const p4 = &x;  
int const* const p5 = &x; // wie p4
```

Konstanter Pointer auf Konstante

Laufzeitkonstanten



Deklaration

- **Referenzen** sind inhärent Konstanten, da sie nach Initialisierung mit einer Variable (=Bindung) nicht mehr eine andere Variable referenzieren können
→ Die Angabe des **const** Kennzeichners ist nur für den referenzierten Datentyp sinnvoll

```
int x = 5;  
int y = 42;  
  
int& r1 = x; // 'r1' ist eine inhärent-konstante Referenz  
           // auf einen veränderlichen Integer  
  
r1 += 5; // OK: Integer kann verändert werden  
r1 = y;  // OK: Integer wird der Wert von 'y' zugewiesen  
           // VORSICHT: Keine Zuweisung der Referenz!
```

Referenz auf veränderliches Objekt

```
int x = 5;  
int y = 42;  
  
const int& r2 = x; // 'r2' ist eine inhärent-konstante Referenz  
                // auf eine Integer-Konstante  
  
r2 += 5; // ERROR: Integer kann nicht verändert werden  
r2 = y;  // ERROR: Integer kann nicht zugewiesen werden
```

Referenz auf Konstante

Laufzeitkonstanten

Lesbarkeit



- Konstanten zu deklarieren, trägt erheblich dazu bei, die **Lesbarkeit** des Programmcodes zu verbessern
 - Die Deklaration ist oft optional, sollte aber wenn immer möglich vorgenommen werden

```
void processInt(const int* i) { ... }
```

```
int main()  
{
```

```
    int i = 5;
```

```
    processInt(&i);
```

```
    ...
```

```
}
```

Deklaration des Funktionsparameters als Pointer auf eine Integer-Konstante

Auch ohne die konkrete Implementierung von `processInt(&i)` zu kennen, kann folgender Programmcode annehmen, dass die Variable `i` nicht verändert wurde!

Laufzeitkonstanten

Konstantenkorrektheit



- Die korrekte Definition von Konstanten zieht sich durch das *gesamte* Projekt; Dies wird als „**Konstantenkorrektheit**“ (**const correctness**) eines Projekts bezeichnet
 - Nicht-veränderbare Objekte, Referenzen und Zeiger sollten frühzeitig **const** deklariert werden, um spätere große Überarbeitungen zu verhindern
 - Zeiger und Referenzen sollten wenn möglich **const** deklariert werden, um Programmierfehler zu vermeiden und die Lesbarkeit von Funktionen zu verbessern

Laufzeitkonstanten



Zusammenfassung

- Laufzeitkonstanten werden mit dem Kennzeichner **const** definiert und tragen erheblich zur **Vermeidung von Programmierfehlern** und der **besseren Lesbarkeit** von Programmcode bei
- Die **Platzierung** des Kennzeichners **const** ändert die Interpretation eines Variablentyps stark, besonders bei Zeigervariablen
- Laufzeitkonstanten sollten von **Beginn der Entwicklung** an definiert werden, um spätere große Überarbeitungen des Projekts zu vermeiden
- Ein Programm, das jeden nicht-veränderlichen Wert **const** deklariert, wird als „**konstantenkorrekt**“ (**const correct**) bezeichnet

Laufzeitkonstanten

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Eine Laufzeitkonstante ist eine Variable, die während des Ablaufs eines Programms nicht verändert werden kann.
- Die Deklaration einer Variablen mit dem `const` Kennzeichner schützt sie vor (ungewollten) Änderungen.
- Ist ein Zeiger `const`-deklariert, ist auch das adressierte Objekt `const`.
- Referenzen sind auch ohne Angabe des `const` Kennzeichners Konstanten.
- Referenzierte Objekte sind auch ohne Angabe des `const` Kennzeichners Konstanten.
- Wird der `const` Kennzeichner vor die Deklaration einer Referenz geschrieben bedeutet das, dass die Referenz nach ihrer ersten Zuweisung nicht mehr verändert werden darf.
- Die Position des `const` Kennzeichners kann bei der Deklaration einer Zeigervariablen frei gewählt werden.

■ Welche Konstanten entstehen durch die Deklaration?

```
int const* ptr;
```

- Konstanter Zeiger
- Konstantes Objekt
- Keine Konstanten
- Konstanter Funktionsparameter



Typumwandlung

Lernziele

- Das Verhalten von Umwandlungen zwischen Datentypen wurde verstanden
 - Implizite Umwandlungen können erkannt werden
 - Explizite Umwandlungen können mit Hilfe von modernen Casts umgesetzt werden



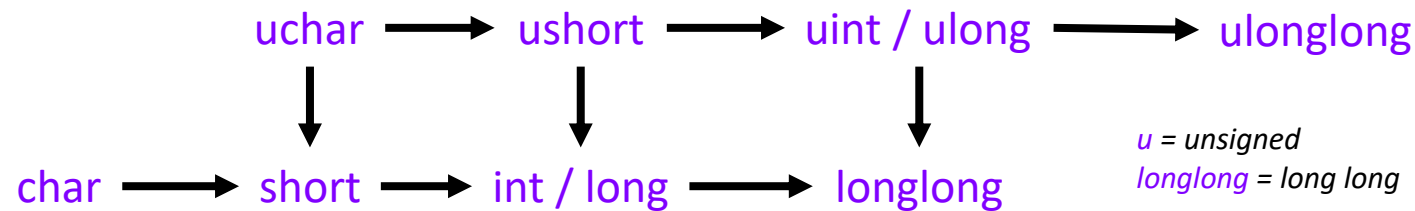
- Die **Typumwandlung** ist die Konvertierung des Werts eines Datentyps in den eines anderen
- Es wird zwischen impliziten und expliziten Typumwandlungen unterschieden
 - **Implizit:** Der Typ wird ohne weitere Angaben durch Zuweisung umgewandelt
 - **Explizit:** Die Umwandlung wird explizit angegeben
- **Implizite** Umwandlungen sind nur zwischen wenigen vorher-definierten Datentypen möglich, z.B.:
 - Zwischen **elementaren Datentypen**,
 - Zeiger auf **Nullpointer** oder **void-Pointer**,
 - Zeiger auf **Arrays & Funktionen**,
 - Objekte, die einen Umwandlungs-**Operator** oder -**Konstruktor** besitzen (Ausblick: Klassen),
 - Zeiger und Referenzen auf **Elternklassen** (Ausblick: Klassen)

Typumwandlung

Elementare Datentypen



- Elementare Datentypen können implizit umgewandelt werden
- Die Umwandlung ist sicher, sofern **kein Datenverlust** entsteht
 - Dies ist der Fall bei Umwandlungen von „kleineren“ zu „größeren“ Datentypen, bei denen der Wertebereich nicht abgeschnitten wird:



- Weiter können integrale Datentypen implizit zu **Gleitkommazahlen** umgewandelt werden
 - Hierbei kann jedoch ein Datenverlust durch die limitierte Präzision von Gleitkommazahlen entstehen

Typumwandlung

Casts

- Explizite Typumwandlungen werden mit **Casts** ausgeführt
- Casts weisen den Compiler an, den Wert eines bestimmten Typs zu konvertieren, auch wenn die Konvertierung nicht sicher ist
 - Löst in einigen Fällen einen Fehler aus, wenn beide Typen überhaupt nicht miteinander verknüpft sind
 - In anderen Fällen wird kein Fehler ausgelöst, selbst wenn der Vorgang nicht Typ-sicher ist
 - Casts sind eine potenzielle Quelle für Programmierfehler
- Casts sind manchmal unvermeidbar
 - Nicht alle Typumwandlungen sind gleichermaßen gefährlich
 - Sollten nur verwendet werden, wenn man genau weiß, was man tut und die Konvertierung nicht zu ungewollten Fehlern oder Programmabstürzen führt



```
// Wert vom Typ double
double pi = 3.14159;

// Expliziter Cast zu int
int castedPi = static_cast<int>(pi);

std::cout << "Originalwert (double): " << pi;
std::cout << "Casted-Wert (int): " << castedPi;
```

Ausgabe:

```
Originalwert (double): 3.14159
Casted-Wert (int): 3
```

Nachkommastellen durch
Cast abgeschnitten

Typumwandlung

Casts



- In C++ gibt es fünf verschiedene Arten von Casts
 - **C-style Cast** `(int)x`
Umwandlungsoperator für alle Arten von Umwandlungen im C-Format
 - **Static Cast** `static_cast<int>(x)`
Für Umwandlungen zwischen ähnlichen Datentypen, wie den elementaren Datentypen
 - **Const Cast** `const_cast<int>(x)`
Zum Entfernen bzw. Hinzufügen des Konstantenstatus auf einem Objekt
 - **Reinterpret Cast** `reinterpret_cast<int>(x)`
Für Umwandlungen zwischen fundamental verschiedenen Datentypen
 - **Dynamic Cast** `dynamic_cast<int>(x)`
Für Umwandlungen in Klassenhierarchien (Ausblick: Klassen)

Typumwandlung

Casts – C-style



■ C-style Cast

- Im C-Format wird der gleiche Umwandlungsoperator für alle Arten von Umwandlungen verwendet
- Sind auf einen Blick im Code schwer zu erkennen
- Sehr schwer vorhersehbar, was bei einer Umwandlung tatsächlich geschieht

```
(int)x; // old-style cast, old-style syntax  
int(x); // old-style cast, functional syntax
```

```
int i = 5;  
float f1 = i / 2;  
float f2 = (float)i / 2;
```

Warnung: „Initialisierung“:
Konvertierung von „int“ zu „float“,
möglicher Datenverlust

Ausgabe:

```
f1: 2, f2: 2.5
```


Typumwandlung

Casts – Static



■ Static Cast

- Mit ***static_cast*** werden Umwandlungen während der Kompilierung überprüft
- Gibt einen Fehler zurück, wenn der Compiler erkennt, dass zwischen inkompatiblen Typen umgewandelt werden soll
- Kann den Konstantenstatus von Objekten nicht ändern
- Kann keine fundamental verschiedenen Typen umwandeln

```
int i = 5;
float f = static_cast<float>(i) / 2;           // wie „float(i) / 2“ oder „(float)i / 2“

const int* p = &i;
int* p2 = static_cast<int*>(p);                // ERROR C2440: "static_cast" kann "const" nicht entfernen

double d = 3.1415925;
std::string s = static_cast<std::string>(d);  // ERROR C2440: "static_cast":
                                              // "double" kann nicht in "std::string" konvertiert werden
```

Typumwandlung

Casts – Const



■ Const Cast

- Mit ***const_cast*** wird der **const** Kennzeichner auf einem Objekt entfernt oder hinzugefügt – also ein Objekt veränderlich oder nicht-veränderlich gemacht = *Konstantenstatus*
- Kann keine anderen Typumwandlungen durchführen
- Wird meist nur zum Entfernen des **const** Kennzeichners verwendet
 - Da das Hinzufügen des **const** Kennzeichners implizit ist
- Meist nur nötig, um mit veralteten Bibliotheken zu interagieren, die nicht konstantenkorrekt sind
 - Zeugt sonst von unsauberem Programmierstil

```
const int i = 5;  
const int* p = &i;  
int* p2 = const_cast<int*>(p); // entfernt „const“, wäre mit static_cast nicht möglich gewesen
```

Typumwandlung

Casts – Reinterpret



■ Reinterpret Cast

- ***reinterpret_cast*** wird für Umwandlungen zwischen fundamental verschiedenen Typen verwendet
- Der Programmierer garantiert dabei, dass die Umwandlung auf der jeweiligen Architektur funktioniert, da der Compiler dies (anders als bei ***static_cast***) nicht überprüft
- Beispiele:
 - Umwandlung von einem Zeiger auf einen **int32** (nur sicher auf 32-Bit Prozessorarchitekturen)
 - Umwandlung von einem Zeiger auf einen **float** zu einem Zeiger auf einen **int** (andere Arithmetik)
- Kann den Konstantenstatus von Objekten nicht ändern

```
int i = 5;
int* p = &i;
int32_t p2 = reinterpret_cast<int32_t>(p); // Pointer → 32-Bit Integer, mit static_cast nicht möglich
                                           // nur auf 32-Bit Prozessorarchitekturen anwendbar

float* p3 = reinterpret_cast<float*>(p);    // Integer Pointer → Gleitkommazahl Pointer, mit static_cast nicht möglich
*p3 += 0.5f;                               // Aufaddieren einer Gleitkommazahl auf eine Ganzzahl, zerstört Ganzzahl
```

Typumwandlung

Zusammenfassung



- Datentypen können **implizit** rein durch Zuweisung umgewandelt werden
 - Ist nur bei wenigen vorher-definierten Datentypen möglich
- Datentypen können **explizit** durch Casts umgewandelt
 - ***static_cast*** sollte gegenüber anderen Casts bevorzugt werden
 - Außer bei Downcasts in Klassenhierarchien (Ausblick: Klassen)
 - Für die Interaktion mit veralteten Bibliotheken kann ***const_cast*** notwendig sein, um den Konstantenstatus auf einem Objekt zu entfernen
 - In Ausnahmefällen kann ***reinterpret_cast*** verwendet werden, um fundamental verschiedene Typen umzuwandeln
 - ***C-style*** Casts wurden vollständig durch moderne C++ Casts ersetzt und sollten nicht mehr verwendet werden
- Komplexere Datentypen, wie Klasseninstanzen und Zeiger / Referenzen auf diese, erfordern weitere Überlegungen bezüglich der Typumwandlung (Ausblick: Klassen)

Typumwandlung

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Unter einer „expliziten Typumwandlung“ versteht man eine Umwandlung, die man selbst angibt.
- Implizite Typumwandlungen sind zwischen allen Datentypen möglich.
- Bei Typumwandlungen zwischen elementaren Datentypen entsteht nie Datenverlust.
- C-Style Casts lösen immer Fehler aus, wenn inkompatible Typen ineinander umgewandelt werden.
- Zur Umwandlung ähnlicher Datentypen sollte **static_cast** genutzt werden.
- Umwandlungen mit **reinterpret_cast** werden vom Compiler auf Gültigkeit überprüft.

■ Bei welchen der folgenden Typumwandlungen kann ein Datenverlust entstehen?

```
int x = ...;  
auto y = (long long)x;
```

```
char x = ...;  
auto y = (unsigned char)x;
```

```
unsigned short x = ...;  
auto y = (long)x;
```

```
unsigned long long x = ...;  
auto y = (long long)x;
```



Bedingungen & Schleifen

Lernziele

- Die Kontrollstrukturen **If-Else**, **Switch**, **For** und **While** wurden verstanden
- Iterative und rekursive Funktionen können kriterienbasiert eingesetzt werden

Bedingungen & Schleifen

Allgemein



Bedingungen

- **If-Else-Statements**
 - Fallunterscheidung nach True-False-Aussagen
 - Erlaubt Deklaration von Variablen
 - **C++17** Mit `if(bool b = true; b)`
- **Switch-Statements**
 - Fallunterscheidung nach Wert einer Variable
 - Funktioniert für Integer-Datentypen (auch char und enum)
 - Nutze *break* oder *return*, wenn ein Fall abgeschlossen ist
 - *default* für alle Fälle, die nicht explizit angegeben sind

Schleifen

- **While-Schleife**
 - Wird wiederholt, bis die Bedingung nicht mehr erfüllt ist
 - In der **Do-While**-Schleife wird der Schleifenkörper einmal ausgeführt, bevor die Bedingung getestet wird
- **For-Schleife**
 - Durchläuft die Schleife für eine bestimmte Anzahl an Elementen
 - Erlaubt Deklaration von Variablen
 - Kann nach jedem Durchlauf den Wert von Variablen anpassen und auf eine Endbedingung testen

Bedingungen & Schleifen

If-Else



Bedingungen

■ If-Else-Statements

- Fallunterscheidung nach True-False-Aussagen
- Erlaubt Deklaration von Variablen
 - **C++17** Mit `if(bool b = true; b)`

```
// Gibt zurück, ob der Spieler das Ziel erreicht hat  
bool reachedGoal() { /* ... */ }
```

```
void main() {  
    std::cout << "Du hast das Ziel "  
    << (  
        reachedGoal() // Rufe reachedGoal auf  
        ? "erreicht!" // Für den Fall, dass TRUE returned wurde  
        : "noch nicht erreicht!" // Für den Fall, dass FALSE returned wurde  
    )  
    << '\n';  
}
```

Der Bedingungs-Operator `?` bietet eine einzeilige Kurzschreibweise für eine reine If-Else-Anweisung in Form von: *Abfrage ? Fall, wenn TRUE : Fall, wenn FALSE*

Schleifen

■ While-Schleife

- Wird nicht
- In der Schleife

```
void resolveInput(char direction)  
{  
    if (direction == 'w')  
        moveForward();  
  
    else if (direction == 'a')  
        moveLeft();  
  
    else if (direction == 's')  
        moveBack();  
  
    else if (direction == 'd')  
        moveRight();  
  
    else  
        doNothing();  
}
```


Bedingungen & Schleifen

Switch



Bedingungen

■ If-Else-Statements

- Fallunterscheidung nach True-False-Aussagen
- Erlaubt Deklaration von Variablen
 - **C++17** Mit `if(bool b = true; b)`

■ Switch-Statements

- Fallunterscheidung nach Wert einer Variable
- Funktioniert für Integer-Datentypen (auch char und enum)
- Nutze *break* oder *return*, wenn ein Fall abgeschlossen ist
- *default* für alle Fälle, die nicht explizit angegeben sind

```
void resolveInput(char direction) {  
    switch (direction) {  
        case 'w':  
            moveForward();  
            break;  
        case 'a':  
            moveLeft();  
            break;  
        case 's':  
            moveBack();  
            break;  
        case 'd':  
            moveRight();  
            break;  
        default:  
            doNothing();  
            break;  
    }  
}
```

Bedingungen & Schleifen

While



Bedingungen

```
// Gibt zurück, ob der Spieler das Ziel erreicht hat
bool reachedGoal() { /* ... */ }
// Behandelt die Nutzereingabe ein
void resolveInput(char direction);
// Gibt die vom Nutzer gedrückte Taste zurück
char getKeyPressed() { /* ... */ }

void main()
{
    // Es wird zuerst überprüft,
    // ob das Ziel erreicht wurde
    while(!reachedGoal())
    {
        char direction = getKeyPressed();
        resolveInput(direction);
    }
}
```

Schleifen

■ While-Schleife

- Wird wiederholt, bis die Bedingung nicht mehr erfüllt ist
- In der **Do-While**-Schleife wird der Schleifenkörper einmal ausgeführt, bevor die Bedingung getestet wird

```
void main()
{
    // Es wird zuerst der Schleifenkörper durchlaufen
    do {
        char direction = getKeyPressed();
        resolveInput(direction);
    } while(!reachedGoal());
    // und am Ende überprüft,
    // ob das Ziel erreicht wurde
}
```

Bedingungen & Schleifen



For

Bedingungen

```
// remainingTries:  
// Anzahl Versuche, das Ziel zu erreichen  
for ( unsigned remainingTries = 10;  
      remainingTries > 0;  
      --remainingTries  
) {  
    // Verlasse die Schleife frühzeitig,  
    // wenn das Ziel erreicht wurde  
    if (reachedGoal)  
        break;  
    char direction = getKeyPressed();  
    resolveInput(direction);  
}
```

abgeschlossen ist

- *default* für alle Fälle, die nicht explizit angegeben sind

Schleifen

Syntax (Zeilenumbruch nicht notwendig):

```
for (  
    [Deklaration oder Ausdruck vor erstem Schleifendurchlauf];  
    [Abbruchbedingung];  
    [Ausdruck, der nach jedem Schleifendurchlauf ausgeführt wird]  
)
```

bevor die Bedingung getestet wird

■ For-Schleife

- Durchläuft die Schleife für eine bestimmte Anzahl an Elementen
- Erlaubt Deklaration von Variablen
- Kann nach jedem Durchlauf den Wert von Variablen anpassen und auf eine Endbedingung testen

Bedingungen & Schleifen



For

Bedingungen

```
// C++ Datentyp für Zeichenketten
std::string inputChain{"wwddasasba"};
// C++11 Range-For Loop
for (auto direction : inputChain)
{
    // Verlasse die Schleife frühzeitig,
    // wenn das Ziel erreicht wurde
    if (reachedGoal)
        break;

    resolveInput(direction);
}
```

Hinweis: Der Range-For Loop wird in einem späteren Kapitel noch tiefer erläutert. Wichtig ist, dass man die Syntax schon einmal gesehen hat und prinzipiell versteht.

Schleifen

Der **Range-For Loop** iteriert elementweise über ein Objekt. In diesem Beispiel über die einzelnen *chars* in *std::string*. Liest sich: Für jede „direction“ aus „inputChain“.

Syntax:

for (Datentyp Name-des-Elements : Objekt-aus-mehreren-Elementen)

■ For-Schleife

- Durchläuft die Schleife für eine bestimmte Anzahl an Elementen
- Erlaubt Deklaration von Variablen
- Kann nach jedem Durchlauf den Wert von Variablen anpassen und auf eine Endbedingung testen

Bedingungen & Schleifen

Iteration & Rekursion



Iteration

```
int faktultaet_Iterativ(int n)
{
    // Initialisierung
    int iFakn = 1;

    // Schleife
    for (int i = 1; i <= n; ++i)
        iFakn = iFakn * i;

    return iFakn;
}
```

Rekursion

```
int faktultaet_Rekursion(int n)
{
    if (n == 1)
        // Endebedingung
        return 1;

    // Rekursionsschritt
    return n * faktultaet_Rekursion(n - 1);
}
```

Bedingungen & Schleifen

Iteration & Rekursion



Iteration

- Sich wiederholende Schleife
- **Vorteile:**
 - Benötigt keinen Funktions-Overhead
 - Schneller und effizienter als Rekursion
 - Einfachere Optimierungsmöglichkeiten
- **Nachteile:**
 - Umständlichere und weniger intuitive Umsetzung bei Algorithmen auf Bäumen und Graphen
 - Informationsweitergabe zwischen Iterationsschritten ist nicht immer Straight-Forward

Rekursion

- Sich selbst aufrufende Funktion
- **Vorteile:**
 - Einfache Umsetzung für Probleme, deren Lösung von kleineren, ähnlichen Problemen abhängt
 - Kürzerer und verständlicherer Code
 - Nutzt Funktionsargumente und Rückgabewerte aus
- **Nachteile:**
 - Jeder Funktionsaufruf benötigt zusätzlichen Speicherplatz
 - Kann bei häufiger Wiederholung schnell zum Programmabsturz führen

Bedingungen & Schleifen

Iteration & Rekursion



CRITERIA	RECURSION	VS	ITERATION
Definition	When a function calls itself directly or indirectly.		When some set of instructions are executed repeatedly.
Implementation	Implemented using function calls.		Implemented using loops.
Format	Base case and recursive relation are specified.		Includes initializing control variable, termination condition, and update of the control variable.
Current State	Defined by the parameters stored in the stack.		Defined by the value of the control variable.
Progression	The function state approaches the base case.		The control variable approaches the termination value.
Memory Usage	Uses stack memory to store local variables and parameters.		Does not use memory except initializing control variables.
Infinite Repetition	It will cause stack overflow error and may crash the system if the base case is not defined or is never reached.		It will cause an infinite loop if the control variable does not reach the termination value.
Code Size	Recursive code is generally smaller and simpler.		Iterative code is generally bigger.
Overhead	Possesses overhead of repeated function calls.		No overhead as there are no function calls.
Speed	Slower in execution.		Faster in execution.

■ Gängige Beispiele zur Verdeutlichung der Unterschiede

- Fakultät
- Fibonacci
- Türme von Hanoi

■ Abwägung zwischen Performance und Verständlichkeit besonders relevant bei Bäumen & Graphen

- Traversieren
- Erkennen von Zyklen

Quelle:

www.interviewkickstart.com/learn/difference-between-recursion-and-iteration

Bedingungen & Schleifen



Zusammenfassung

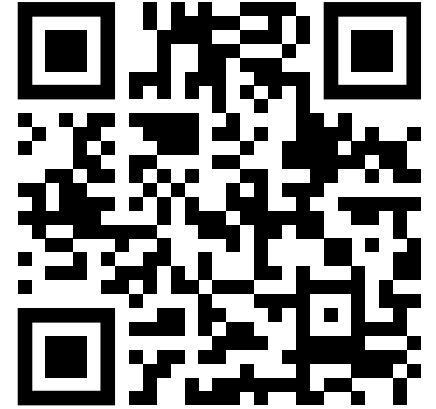
- Mit **If-Else-Statements** werden abhängig von Bedingungen Codeblöcke ausgeführt
- Mit **Switch-Statements** werden abhängig von Werten einer Variable Codeblöcke ausgeführt
- Eine **While-Schleife** wiederholt einen Codeblock so lange, bis die Bedingung *FALSE* ist
 - Sie kann **Kopf-gesteuert** (`while (...) { ... }`) oder **Fuß-gesteuert** (`do { ... } while (...) ;`) sein
- Eine **For-Schleife** wiederholt einen Codeblock für einen bestimmten Wertebereich (Zählervariable, Elemente eines Containers, ...)
- Algorithmen können häufig **iterativ** und **rekursiv** umgesetzt werden
 - **Iterativ**: Verwendung einer Schleife in einer Funktion
 - **Rekursiv**: Mehrmaliger Aufruf der Funktion ohne Schleifen

Bedingungen & Schleifen

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Ein If-Else-Statement kann maximal zwei mögliche Fälle behandeln.
- Ein Switch-Statement kann Werte eines **std::string** unterscheiden.
- Fälle von If-Else-Statements können mit Codeblöcken (Gültigkeitsbereichen) abgegrenzt werden.
- Eine While-Schleife wird immer mindestens einmal ausgeführt.
- Eine For-Schleife eignet sich besonders gut, wenn die Anzahl der Durchläufe vorab bekannt ist.
- Die break-Anweisung beendet eine Schleife oder ein Switch-Statement sofort.
- Eine iterative Funktion ist eine Funktion, die sich selbst aufruft.
- Eine rekursive Funktion muss mindestens eine Endbedingung definieren.



Weitere Grundlagen

Lernziele

- Weitere, für diese Veranstaltung relevante, Grundlagen wurden oberflächlich verstanden

Weitere Grundlagen



Struct

- Ein **struct** ist eine Datenstruktur, mit der zusammengehörende Objekte organisiert werden können
 - Beliebige Variablen beliebiger Datentypen können zusammengesetzt werden
 - Die Variablen werden zu einer Einheit zusammengesetzt
 - Seit **C++11** können Variablen eines structs default-initialisiert werden

```
struct ArrayWithSize
{
    size_t size = 0;
    double* elem = new double[size];
};
```

Definition eines structs mit default-Initialisierern

```
int main()
{
    // Initialisierung des Arrays
    ArrayWithSize a{ 10 };
    for (size_t i = 0; i < a.size; ++i)
        a.elem[i] = i;
    delete[] a.elem;
}
```

a.elem wird ohne explizite
Angabe default-initialisiert

Nutzung eines Objekts des structs

Weitere Grundlagen



Union

- Ein **union** organisiert Objekte, die alle auf den gleichen Speicherplatz zugreifen
 - Die Objekte teilen sich den gleichen Speicherbereich
 - Das Union ist so groß, wie das größte Objekte, das darin enthalten ist
 - Sehr Fehleranfällig durch falsche Interpretation der Objekte
 - Wird in der Praxis kaum verwendet

```
union IntOrChar
{
    int i;
    char c;
};
```

Definition eines Union, das entweder einen Integer oder einen Character darstellt

```
int main()
{
    IntOrChar x;

    x.i = 63; // Setzen des Integers
    std::cout << "i: " << x.i << ", c: " << x.c; i: 63, c: ?

    x.c = 'a'; // Setzen des Characters
    std::cout << "i: " << x.i << ", c: " << x.c; i: 97, c: a
}
```

Nutzung der Objekte eines Unions

Weitere Grundlagen



Enumerationen

- Eine **Enumeration** („Aufzählung“ / „Nummerierung“) enthält eine Menge von benannten numerischen Konstanten (= Zählern), die anhand ihrer Definitionsreihenfolge aufsteigend nummeriert werden
 - Der Name der Konstante kann austauschbar mit dem numerischen Wert verwendet werden
- Eine Enumeration wird mit dem Keyword „**enum**“ deklariert

Zähler
(enumerators)
Konstanten der
Enumeration **Color**

```
enum Color
{
    black, // 0
    white, // 1
    red    // 2
};

Color color = red;
std::cout << "Color is: " << color;
```

Ein Zähler entspricht einem
numerischen Wert
→ Numerische Ausgabe

Color is: 2

Definition eines enums mit drei Konstanten

Weitere Grundlagen

Enumerationen – Unscoped



- Ohne weitere Zusätze zu dem Keyword „enum“ ist eine Enumeration „**unscoped**“
 - Die Zähler sind nicht in dem Bereich der Deklaration (= scope) begrenzt oder gebunden
 - Die Zähler werden implizit in integrale Datentypen umgewandelt
 - Der **zugrundeliegende Datentyp** eines Zählers ist nicht definiert und wird vom Compiler bestimmt
 - Ein zugrundeliegender Datentyp kann jedoch explizit definiert werden
 - Enumerationen können nur bedingt vorausdeklariert werden (Ausblick: Klassenbeziehungen)

```
int main()
{
    enum Color
    {
        black,
        white,
        red
    };

    Color color = black; // Ok
    int color2 = black; // Ok

    bool white = false; // Compilerfehler
}
```

Deklaration eines Unscoped-Enums

Trotz des endenden Deklarationsbereichs, sind die Zähler black, white, red weiterhin sichtbar

Durch fehlende Bereichsbegrenzung kommt es zur Namenskollision zwischen dem Variablen-Name white und dem Zähler-Name white

Weitere Grundlagen

Enumerationen – Scoped

- **C++11** Enumerationen, welche mit der Keyword-Folge „**enum class**“ oder „**enum struct**“ definiert werden, sind **scoped**
 - Die Zähler sind in dem Bereich der Deklaration (= scope) begrenzt und gebunden
 - Die Zähler werden nicht implizit in integrale Datentypen umgewandelt
 - Der **zugrundeliegende Datentyp** eines Zählers ist definiert als **int32**
 - Kann weiterhin explizit definiert werden
 - Scoped Enumerationen können immer vorausdeklariert werden (Ausblick: Klassenbeziehungen)

Die Zusätze **class** und **struct** sind exakt äquivalent und können nach Belieben gewählt werden. Es existieren keine semantischen Unterschiede zwischen ihnen.



```
int main()
{
    enum class Color
    {
        black,
        white,
        red
    };

    Color color = black; // Compilerfehler
    Color color2 = Color::black; // Ok
    int color3 = Color::black; // Compilerfehler
    int color4 = static_cast<int>(Color::black); // Ok

    bool white = false; // Ok
}
```

Deklaration eines Scoped-Enums

Die Enumeration muss vor dem Zähler angegeben werden

Keine Namenskollision zwischen dem **Variablen-Name** white und dem **Zähler-Name** white

Weitere Grundlagen

Enumerationen – Best Practice



- Scoped-Enums sollten bevorzugt werden, da sie **strenger typisiert** sind
 - Sie „verschmutzen“ den Code durch globale Namensdefinitionen nicht
 - Sie unterscheiden sich klar von numerischen Werten, die berechnet werden können

Unscoped-Enum

```
int main()
{
    enum Color { black, white, red };

    Color color = red;

    if (color > 1.5)
    {
        std::cout << square(color);
    }
}
```

Vergleich zwischen dem Zähler einer Farbe und einer Kommazahl (enum → int → double)

Quadrat des Zählers einer Farbe (enum → int)

Scoped-Enum C++11

```
int main()
{
    enum class Color { black, white, red };

    Color color = Color::red;

    if (color > 1.5)
    {
        std::cout << square(color);
    }
}
```

Compilerfehler
(ohne explizite Casts)

```
✗ C2676  binary '>': 'main::Color' does not define this operator or a conversion
✗ C2664  'int square(int)': cannot convert argument 1 from 'main::Color' to 'int'
```

→ Weniger Programmierfehler durch versteckte Umwandlungen

Weitere Grundlagen

Operatoren



- Operatoren definieren eine Operation zwischen zwei Objekten/Operanden

Beispiel: Vergleichsoperator

```
int x = 1;  
int y = 2;  
bool res = x > y;
```

Operator, der Objekt „x“ mit Objekt „y“ vergleicht

- Es existieren vier Arten von Operatoren
 - Zuweisungsoperatoren `x = y`
 - Arithmetische Operatoren
 - Vergleichsoperatoren
 - Logische Operatoren

Weitere Grundlagen

Operatoren



Arithmetische Operatoren			Vergleichsoperatoren			Logische Operatoren		
x+y	Addition*	1+2=3	x==y	Gleichheit	5==6 <i>false</i>	x&y	Bitwise Und*	11b&10b <i>10b</i>
+x	Unäres Plus	+(-3) = -3	x!=y	Ungleichheit	5!=6 <i>true</i>	x y	Bitwise Oder*	11b 10b <i>11b</i>
x-y	Subtraktion*	3-2=1	x<y	Kleiner als	5<5 <i>false</i>	x^y	Bitwise XOR*	11b^10b <i>01b</i>
-y	Unäres Minus	-(-3) = -3	x>y	Größer als	5>5 <i>false</i>	~x	Bitwise Komplement	~10b <i>01b</i>
x*y	Multiplikation*	2*3=6	x<=y	Kleiner gleich	5<=5 <i>true</i>	x&&y	Logisches Und	true&&false <i>false</i>
x/y	Division*	6/3=2	x>=y	Größer gleich	5>=5 <i>true</i>	x y	Logisches Oder	true false <i>true</i>
x%y	Modulo*	7%3=1				!x	Logisches Nicht	!true <i>false</i>

* Die meisten binären Operatoren können mit einem Zuweisungsoperator kombiniert werden: $x+=y \Leftrightarrow x=x+y$

Weitere Grundlagen

Operatoren



- **Schiebeoperatoren** verschieben die Bits des ersten Operanden um eine Anzahl von Positionen
 - Beide Operanden müssen ganzzahlige Werte sein
 - Ist nicht definiert, wenn der zweite Operand negativ ist oder größer/gleich der Bitanzahl des Datentyps des ersten Operanden
- Der **Links-Shift** (<<) verschiebt die Bits nach links
 - frei werdende Bits werden zur 0
 - $a \ll x$ ist identisch mit der Multiplikation von a mit 2^x

```
void printByte(uint8_t byte) {  
    for (uint8_t i = 0; i < 8; ++i)  
        printf("%d", !((byte << i) & 0x80));  
}  
printByte(0b0011'1011);
```

i	byte << i	(byte << i) & 0x80	!((byte << i) & 0x80)
0	0011'1011	0000'0000	0 (bool)
1	0111'0110	0000'0000	0 (bool)
2	1110'1100	1000'0000	1 (bool)
3	1101'1000	1000'0000	1 (bool)
4	1011'0000	1000'0000	1 (bool)
5	0110'0000	0000'0000	0 (bool)
6	1100'0000	1000'0000	1 (bool)
7	1000'0000	1000'0000	1 (bool)



Weitere Grundlagen

Operatoren

- **Schiebeoperatoren** verschieben die Bits des ersten Operanden um eine Anzahl von Positionen
- Der **Rechts-Shift** (>>) verschiebt die Bits nach rechts
 - Für $a \gg x$ gilt:
 - Die letzten x Bits von a werden gestrichen
 - Ist a **positiv**, werden vor a x Nullen aufgefüllt
 - Ist a **negativ**, werden vor a x Einsen aufgefüllt
 - identisch mit dem der abgerundeten Division durch 2^x : $a \gg x \triangleq \left\lfloor \frac{a}{2^x} \right\rfloor$

```
int8_t a1 = 0b0101'1010; // 90
int8_t a2 = 0b1010'0110; // -90
uint8_t a3 = 0b1111'1010; // 250
int8_t a4 = 0b1111'1010; // -6

printf("a1 >> 2: %d\n", a1); 00010110: 22
printf("a2 >> 2: %d\n", a2); 11101001: -23
printf("a3 >> 3: %d\n", a3); 00011111: 31
printf("a4 >> 3: %d\n", a4); 11111111: -1
```

Weitere Grundlagen

Konsolenausgabe



■ **printf** (*Funktion*)

- Formatierte Konsolenausgabe
- %[flags][width][.precision][length]specifier
- *specifier* interpretieren den gegebenen Wert z.B. als Character (**c**), signed decimal integer (**d**), unsigned decimal integer (**u**), unsigned hexadecimal integer (**x**) oder Gleitkommazahl (**f,g,e,a**)
- Die optionalen Argumente spezifizieren die Darstellung des Wertes

<code>printf("%d\n", 10);</code>	10
<code>printf("%3.2f", 13.256);</code>	13.26
<code>printf("123456789\n");</code>	123456789
<code>printf("%8g\n", 3.14);</code>	3.14
<code>printf("%5g\n", 3.14);</code>	3.14

■ **std::cout** (*Operator (überladen)*)

- Aus der Bibliothek `<iostream>`
- Ausgabestream für die Konsole
- Ermöglicht Manipulation der Formatierung auf unterschiedliche Arten
 - **C++20** z.B. über die Bibliotheksfunktion `format()` aus dem Header `<format>`; mehr dazu in einer späteren Vorlesung

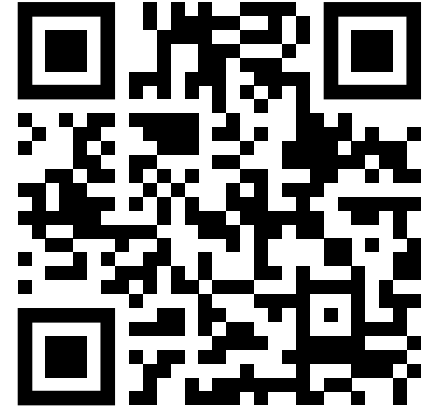
<code>std::cout << 10 << std::endl;</code>	10
<code>std::cout << -2.78 << std::endl;</code>	-2.78
<code>std::cout << "H" << "i" << '\n';</code>	Hi
<code>std::cout << std::format("{:3.2f}\n", 13.256);</code>	13.26

Weitere Grundlagen

Verständnisfragen

PIN: 469452

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Objekte eines structs belegen alle den gleichen Speicherplatz.
- Enumerationen, die allein mit dem Keyword „enum“ deklariert werden, sind *scoped*.
- In einem *scoped* enum sind die Zählerkonstanten an den Bereich der Deklaration gebunden.
- Zählerkonstanten eines *scoped* enums werden implizit in numerische Datentypen umgewandelt.
- Schiebeoperatoren manipulieren die Bits eines Objekts.
- Die Konsolenausgabe mit dem << Zeichen ist eine Funktion.